University of Padua

Department of Mathematics "Tullio Levi-Civita"

Bachelor's Degree in Computer Science

Explainable Machine Learning Through Large Language Models: Analysis and Prompt Design

Bachelor's Thesis

Supervisor

Prof. Michele Scquizzato

Candidate

Lorenzo Soligo

ID number 2101057

ACADEMIC YEAR 2025-2026

By far, the greatest danger of Artificial Intelligence is that people conclude too early that they understand it.

— Eliezer Yudkowsky

Abstract

As Machine Learning (ML) models become increasingly prevalent in business applications, ensuring their transparency and reliability through Explainable Artificial Intelligence (XAI) is critical. This project, conducted in collaboration with Zucchetti S.p.A., investigates the interpretability of various machine learning algorithms.

The research systematically evaluates a spectrum of predictive models, ranging from fundamental Regression and Classification models—such as Linear Regression, Logistic Regression, Support Vector Machines, and Decision Trees—to complex ensemble methods, including Random Forest, XGBoost and Symbolic Regression. The analysis explores both intrinsic model transparency and post-hoc explainability techniques, utilizing metrics like SHapley Additive exPlanations (SHAP) to interpret predictions and feature importance. Furthermore, the methodology encompasses the examination of mathematical foundations, internal knowledge representations, and the impact of ensemble techniques on both model performance and interpretability. Real-world validation is performed using datasets such as Student Salary Prediction[1], Life Expectancy (WHO)[2] and Heart Failure Prediction Dataset[3].

A primary objective of this work is bridging the gap between technical accuracy and human-understandable explanations for non-expert stakeholders. To achieve this, the project leverages advanced Prompt Engineering principles. Tailored prompts for Large Language Model (LLM) are developed for each analyzed algorithm. These prompts are explicitly designed to automatically generate human-readable explanations of algorithmic behaviors and predictions. Ultimately, the project delivers production-ready implementations and LLM integration adapters, demonstrating how the synthesis of traditional XAI methodologies with modern language models can significantly enhance the transparency, trust, and responsible deployment of Artificial Intelligence (AI) systems.

Acknowledgements

Firstly, I would like to thank Prof. Michele Scquizzato for their guidance and support during the writing of this thesis and throughout the duration of the internship. I would also like to express my gratitude to the team at Zucchetti for the opportunity to work on this project and to gain this valuable experience.

Furthermore, I would like to thank my friends and family for their encouragement throughout my university years.

Special thanks to Filippo, Nicola, and Riccardo, who were always there to share the ups and downs, and to Alessia, for her love and constant support.

Padua, June 2026 *Lorenzo Soligo*

Contents

List of Figures

List of Tables

Chapter 1

Introduction

This chapter provides an introduction to the internship concept, goals, company, and thesis organization. It is meant to provide the necessary context for the reader to understand the motivation and the structure of the thesis.

1.1 Company

Zucchetti S.p.A. is a leading Italian software company specializing in providing interpretable IT solutions for businesses. Founded in 1978, Zucchetti has become one of Italy's largest software providers, offering a wide range of products and services that cater to various industries, including manufacturing, retail, healthcare, and public administration.

With a strong focus on innovation and customer satisfaction, Zucchetti is committed to helping organizations optimize their operations and achieve their business goals through cutting-edge technology. Driven by this commitment, the company has been actively involved in the development and implementation of ML and AI solutions, aiming to enhance the capabilities of their software offerings and provide more value to their clients.

Zucchetti S.p.A. logo.

1.2 Internship concept

The internship project is centered around the exploration of XAI techniques, with a specific focus on the interpretability and comprehensibility of machine learning algorithms. The primary objective is to analyze a variety of predictive models, ranging from basic regression and classification algorithms to more complex ensemble methods, in order to evaluate their transparency (interpretability) and explainability. The idea is to increase the understanding of the results of the models, and to make them more accessible to non-expert stakeholders, by leveraging advanced Prompt Engineering (PE) principles to develop tailored prompts for LLMs that can automatically generate human-readable explanations of algorithmic behaviors and predictions.

The timeline of the project is structured as follows:

Week	Task
1st	Analysis of Linear Regression, Logistic Regression, Support Vector Machines.
2nd	Analysis of Decision Trees, Random Forests, XGBoost.
3rd	Code implementation and Prompt Engineering for the analyzed algorithms.
4th	Evaluation and documentation of the results.
5th	Deep dive in Random Forests and research of real-world dataset.
6th	Analysis of Symbolic Regression.
7th	Code implementation and Prompt Engineering for the analyzed algorithms.
8th	Evaluation and documentation of the results, final report writing.

Table of the project timeline.

1.3 Project goals

The goals follow a notation to distinguish between necessary (N), desirable (D), and optional (O) goals. In the planning phase, the goals were defined as follows:

Goal	Description
N-01	Complete and profound understanding of the chosen algorithms, their mathematical foundations and their internal knowledge representation.
N-02	Comprehension of interpretability and explainability techniques in machine learning.
N-03	Comprehension of algorithms design techniques.
N-04	Prompt generation for Large Language Models to generate human-readable explanations.
N-05	Comprehension and application of Prompt Engineering principles in the context of XAI.
D-01	Creation of a metric to evaluate the interpretability and explainability of the analyzed algorithms.
D-02	Automatization of the analysis pipeline from dataset to LLM requests.
O-01	Application in real-world scenarios of the interpretability and explainability of Machine Learning models.
O-02	Interpretability and explainability study of semi-supervised and unsupervised algorithms.

Table of project goals.

1.4 Thesis structure

The second chapter describes the basic concepts that fuel the project. The concepts spread from the mathematical foundations of the analyzed algorithms, the basics of interpretability and explainability, to the principles of Prompt Engineering and LLMs.

The third chapter analyzes the ML algorithms, focusing on their capability, limitation and interpretability.

The fourth chapter describes the design and implementation of the analysis pipeline from dataset to LLM requests.

The fifth chapter discusses prompt engineering techniques and their application in the context of XAI.

The sixth chapter presents the conclusions of the study, the findings and the goals assessment.

Chapter 2

Background knowledge

In this chapter, we will provide the necessary background knowledge to understand the project. We will cover the mathematical foundations of the analyzed algorithms, the basics of interpretability and explainability, and the principles of Prompt Engineering and LLMs.

2.1 Algorithmic analysis structure

The analysis of the algorithms is structured systematically to ensure a thorough evaluation of their interpretability and functioning. The structure includes the following components:

1. **Mathematical foundations**: detailed examination of the mathematical principles underlying each algorithm, including their assumptions and theoretical properties. The focus is on the understanding of the specific Objective Function that the algorithms optimize, the regularization techniques they use, and the mathematical properties that govern their behavior.
2. **Computational Complexity**: analysis of the computational requirements of each algorithm, including time complexity and scalability regarding both training and inference.
3. **Internal representation**: exploration of how each algorithm internally represents data and makes decisions, especially regarding Categorical Features.
4. **Data assumptions**: investigation into the assumptions each algorithm makes about the data, such as linearity, independence of features, and distributional assumptions.
5. **Predictive performance and limitations**: evaluation of the predictive performance of each algorithm on relevant datasets, considering the data assumptions and mathematical foundations of the models.
6. **Metrics for prediction quality**: presentation of the most relevant metrics for evaluating the model's performance, based on the task.
7. **Interpretability and explainability metrics**: presentation of the most relevant metrics for evaluating the models' interpretability and explainability, based on the task.
8. **Interpretability and explainability limitations**: discussion of the limitations of both the intrinsic interpretability of the models and the post-hoc explainability techniques applied to them.

2.2 Explainability

Interpretability and explainability of a ML model are key concepts in understanding its behavior. Specifically, **interpretability** refers to the ability to show how the model works, while **explainability** refers to the ability to explain why a specific outcome occurred.
Contrary to intuition, they are **not binary attributes**, but form a spectrum with multiple dimensions.

2.2.1 Interpretability and explainability dimensions

1. **Intrinsic Transparency (Model-Level Interpretability)**
   The model is inherently transparent and interpretable by design. The structure directly communicates how it works, such as regression coefficients or decision tree nodes. Each prediction is fully traceable, and parameters have tangible meanings. Explanations reflect human reasoning, making them coherent from a domain perspective.
   It represents the best-case scenario for interpretability but it is often in trade-off with performance, as more complex models tend to be less transparent but more powerful.
   It is generally organized in tiers: high, medium, and low transparency.
2. **Global Interpretability (Model-Level Interpretability):**
   The model is comprehensible in its entirety, enabling the user to understand its overall behavior and decision-making process. Such a level of understanding is hard to achieve, and it is usually reached indirectly by comprehending the individual components of the model instead of the model as a whole.
3. **Modular Interpretability:** When global interpretability is not achievable, we can still understand the model by analyzing its components separately. Exploiting the model transparency, we can understand the role of each component in the decision-making process.
   This concept is clear in the case of linear methods, where it is easy to understand the contribution of the single weights or in tree based models, where the splits and the structure of the tree can be analyzed separately.
4. **Local Explainability (Instance-Level Explainability):** The model might be opaque and too complex to understand globally. However, for a specific prediction, we can generate an explanation that is locally explainable. SHAP could be used in this context to explain why a particular instance obtained a certain prediction.

2.2.2 Explanation properties and factors that facilitate understanding

For human explanations to be effective, they should have certain properties that facilitate understanding and usability. The following properties can be identified from Robnik-Sikonja and Bohanec 2018 and Molnar [4]:

- **Contrastive explanations**

Comparative explanations that highlight the differences between instances can be more informative than absolute explanations. Comparing instances makes the explanation more impactful, providing a direct idea of how different factors contribute to different outcomes.

- **Cause selection**

Not all features are equally important; explanations should focus on the most influential factors rather than listing all contributing features. A small sample of the most relevant features provide a more clear explanation than a long list of all features preventing information overload.

- **Social and contextual factors**

The social environment and the audience's expertise level should be considered when generating explanations to ensure they are appropriate and understandable.

- **Anomalies or abnormal predictions**

Highlighting anomalies or abnormal predictions can help users understand when the model is making an unusual decision, which can be crucial for trust and debugging. Features that are not largely relevant for the majority of the predictions but that have a great impact on specific predictions should be highlighted for their capability.

- **Fidelity**

The explanation should accurately reflect the model's behavior and not introduce misleading information. A high-fidelity explanation ensures that users can trust the insights provided and make informed decisions based on them, provided that the model is accurate in its predictions.

- **General and probable**

In the absence of anomalies of specific cases, a good explanation should rely on general and probable causes that are likely to be true for most instances. This parameter can be quantified by the feature support:

$$\text{Feature Support}_i = \frac{\#\text{training instances with similar values to } \ x_i}{\#\text{total training instances}}$$

Generality can be misleading, as it may lead to explanations that are too broad and not specific enough to be useful. It's important to balance generality with the impact of the explanation, ensuring that it provides meaningful insights without being overly vague. This is why usually abnormal features are highlighted, as they can provide more specific and impactful explanations.

2.2.3 SHAP analysis

SHAP (SHapley Additive exPlanations) is a method for explaining the output of machine learning models by attributing the prediction to each feature. It is based on the concept of Shapley values from cooperative game theory, which provide a way to fairly distribute the "payout" (in this case, the prediction) among the "players" (the features) based on their contribution to the prediction. SHAP values can be used to understand the importance of each feature for a

specific prediction (local explainability) or for the model as a whole (global interpretability). SHAP provides a unified framework for interpreting various types of models and can be applied to both linear and non-linear models, making it a powerful tool for explainable AI.

SHAP library logo.

2.3 Prompt Engineering and LLMs

The following sections present the principles used in the design of prompts to guide LLMs in generating human-readable explanations of the analyzed algorithms.

2.3.1 Expert Personas

The expert persona technique consists of designing prompts that describe the role the LLM should play in the task. This technique is intended to guide the model in generating a more precise, coherent, and relevant output.

Though using expert personas has been linked to a loss of accuracy in multiple benchmarks in recent studies by Zizhao Hu [5], the primary use of the LLMs in this project is results evaluation and text generation. The data and metrics do not need to be generated; they only need to be read directly from the prompt. The paper specifically addresses tasks that depend on pretrained knowledge retrieval accuracy [5]. Therefore, it is reasonable to conclude that in this context, the potential loss of accuracy is not a critical issue. In the same paper, an alignment of behavior and style to the described personas is observed. This drift in behavior is highly beneficial for describing algorithmic results in a human-readable way, as the style of the explanation is fundamental to making it accessible to non-expert stakeholders.

2.3.2 Familiar formats

The extensive use of LLMs has led to the emergence of some familiar formats that are widely used in prompt design.

The principal features that make these formats particularly effective are:

- Wide use in training: the more a format is represented in the training data of the model, the more likely the model is to effectively interpret content in that format.

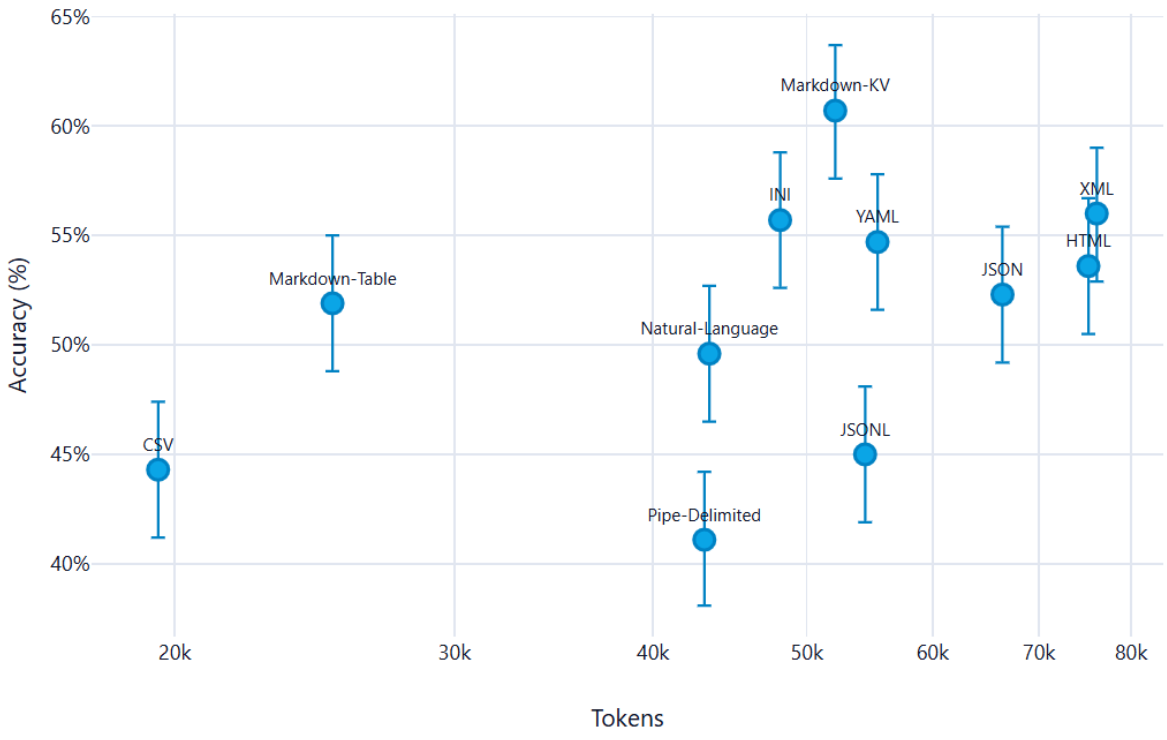
- Clear structure: the format should have a clear and recognizable structure that enables the model to easily identify the different components of the prompt and their relationships.
- Token efficiency: the format should be designed to minimize the number of tokens used while still conveying all necessary information. This is particularly important because token usage directly affects the performance and cost of the model.

Among the many formats available, several have stood out for their accuracy and efficiency. The most valuable of these is **Markdown**, which has a clear, hierarchical structure that allows the model to easily parse the prompt. Using Markdown also enables the creation of a clear and organized structure for the generated explanations, making them more readable and accessible to non-expert stakeholders.

Another accurate format is YAML, which offers a more readable formatting than JSON and XML while still being well-structured.

An alternative like CSV can be used for tabular data, as it is a widely recognized format and is well understood by the model. However, it is important to ensure that the CSV data is properly structured to enable the model to generate accurate and relevant explanations.

Figure 2.5 presents a summary of the eleven formats examined in the study [6]. The y-axis represents accuracy, while the x-axis represents token consumption. It is clear that Markdown emerges as the most accurate format while maintaining good token efficiency.



Formats for prompting. Source: '@formats-llms.'

2.3.3 Chain-of-Thought

Chain of Thought (CoT) prompting is a technique that consists of designing prompts that guide the LLMs to generate a step-by-step reasoning process before reaching the final answer.
The value of this guided, step-by-step reasoning has been proven in multiple benchmarks [7]. This technique is especially valuable in the context of XAI and for non-technical audiences, as it enhances the model's ability to provide detailed and understandable explanations. The generated answer is not just a final result, but an interpretable explanation that breaks down the reasoning process into clear, logical steps.

2.3.4 Zero-Shot prompting

This technique consists of designing prompts that do not include any examples of the expected output. It is the simplest approach to prompting because it eliminates the need to provide training examples. Such a method is necessary in this context because providing specific examples of data and metrics for reference could make the prompt too narrow and prevent generalization. An additional set of data could also confuse the model and bias its output on the actual results of the analysis.
Zero-shot prompting has also been shown to be highly effective when paired with other techniques, such as Chain of Thought (CoT) prompting [8].

2.3.5 Multimodal prompt

The multimodal prompting technique consists of designing prompts that include multiple modalities, such as text, images, and tables. This technique is particularly useful in this project, as it enables providing the model with a richer and more interpretable context for generating explanations.
The use of multimodal prompts can enhance the model's ability to comprehend and analyze the results of the algorithms by leveraging information provided in different formats. In fact, visual prompts interact effectively with textual prompts, enhancing alignment between modalities and thereby improving the model's performance on zero-shot instruction learning [9].

Chapter 3

ML algorithms analysis

In this chapter, we will analyze the machine learning algorithms used in the project, looking at the mathematical foundations, the data requirements, the predictive performance, and the interpretability of the results.
This initial analysis will guide the design and implementation of the pipeline as well as the creation of the specific prompts for the LLMs to generate human-readable explanations of the results.

3.1 Common metrics for algorithm evaluation

To evaluate machine learning algorithm performance in classification and regression tasks, we will use some common metrics paired with the algorithms' specific metrics.

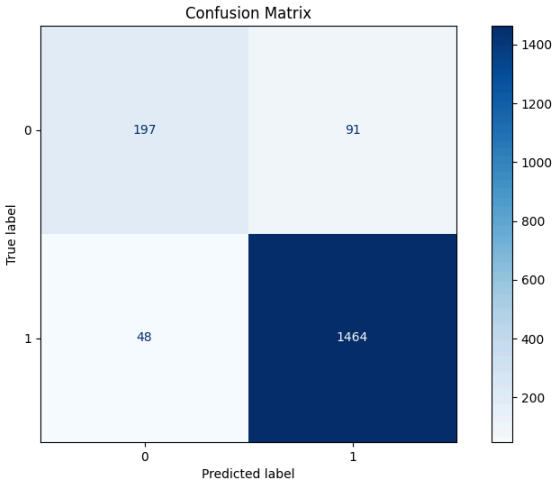
3.1.1 Classification metrics

The following is a list of metrics for evaluating the predictive performance of the logistic regression in the context of the classification task.
Before presenting these metrics, it is important to define some terms, used later:

- **True Positives (TP)**: instances correctly predicted as positive
- **True Negatives (TN)**: instances correctly predicted as negative
- **False Positives (FP)**: instances incorrectly predicted as positive
- **False Negatives (FN)**: instances incorrectly predicted as negative

Confusion Matrix

The confusion matrix is a table that summarizes the results of a classification task by comparing the true class labels with the predicted class labels.
The 4 different categories are TP, TN, FP and FN as described before.



Confusion matrix of a logistic regression model.

Accuracy

Accuracy measures the ratio of correct predictions to the total predictions:

$$\text{ACC} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{TN} + \text{FP} + \text{FN}}$$

It is important to note that in the context of imbalanced classes, accuracy can be misleading, as a model that always predicts the majority class can achieve high accuracy while performing poorly on the minority class.

Sensitivity (Recall / True Positive Rate)

The sensitivity, also known as recall or true positive rate, measures the ratio of correctly predicted positive instances to all actual positive instances:

$$\text{REC} = \text{SENS} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

Specificity (True Negative Rate)

The specificity, also known as true negative rate, measures the ratio of correctly predicted negative instances to all actual negative instances:

$$\text{SPEC} = \frac{\text{TN}}{\text{TN} + \text{FP}}$$

Sensitivity and specificity are particularly important in contexts where the cost of false positives and false negatives is different, such as in medical diagnosis.

Precision

Precision measures the ratio of correctly predicted positive instances to all predicted positive instances:

$$\text{PREC} = \frac{\text{TP}}{\text{TP} + \text{FP}}$$

Precision is crucial in scenarios where the cost of false positives is high, such as in spam detection or fraud detection.

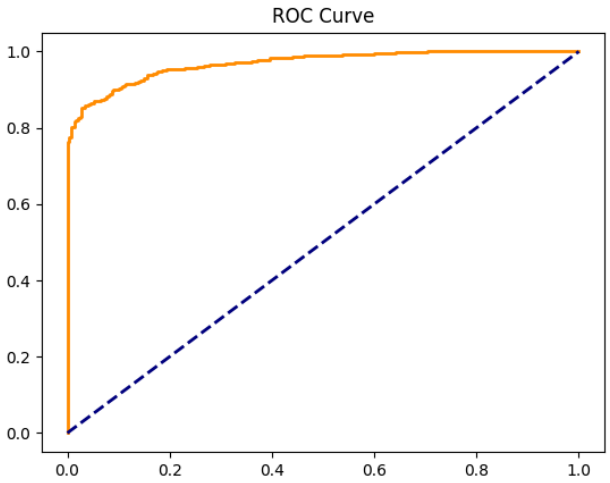
F1-Score

Out of the box, the precision and recall can be in tension, as improving one often leads to a decrease in the other. The F1-score is the harmonic mean of precision and recall, providing a single metric that balances both:

$$\text{F1} = 2\frac{\text{PREC} \cdot \text{REC}}{\text{PREC} + \text{REC}}$$

It is particularly useful in imbalanced classification problems or in situations where both false positives and false negatives are costly.

ROC Curve and AUC



ROC curve of a logistic regression model.

The **ROC Curve** is a visual representation of the trade-off between the True Positive Rate (Sensitivity) and the False Positive Rate as the classification threshold varies. Sensitivity is plotted on the y-axis and the False Positive Rate on the x-axis.
A model with good performance will have a curve that bows towards the top-left corner of the plot, indicating high sensitivity and a low false positive rate across different thresholds. A model that predicts randomly will have a curve that follows the diagonal line.

AUC (Area Under the Curve) is the area under the ROC curve, which numerically quantifies the overall ability of the model to discriminate between positive and negative classes. The AUC ranges from 0 to 1, with higher values indicating better performance and 0.5 representing random guessing.
It can be interpreted as the probability that the model will rank a randomly chosen positive instance higher than a randomly chosen negative instance.

3.1.2 Regression metrics

R^2 (Determination Coefficient)

R^2 quantifies how much the model explains the total variance of the data. It ranges from 0 to 1, where 0 represents a model that cannot explain the data, and 1 represents a perfect adherence.

$$R^2 = 1 - \frac{SSE}{SST}$$

Where:

- $SSE = \sum_{i=1}^{n}(y^{(i)} - \hat{y}^{(i)})^2$ is the Sum of Squared Errors, representing the unexplained variation by the model
- $SST = \sum_{i=1}^{n}(y^{(i)} - \bar{y})^2$ is the Total Sum of Squares, representing the total variation in the target variable

$\bar{R}^2$ (Adjusted R^2)

R^2 tends to increase with the number of features, even if those features are not useful. Adjusted R^2 penalizes the addition of non-informative features.

$$\bar{R}^2 = 1 - (1 - R^2)\frac{n-1}{n-p-1}$$

Where n is the number of instances and p is the number of features.

Root Mean Squared Error (RMSE)

RMSE measures the magnitude of the errors, in the same scale as the target variable:

$$\text{RMSE} = \sqrt{\frac{SSE}{n}}$$

Mean Absolute Error (MAE)

MAE measures the average magnitude of the errors, without considering their direction:

$$\text{MAE} = \frac{1}{n}\sum_{i=1}^{n}| \ y_i - \hat{y}_i \ |$$

3.2 Linear regression

3.2.1 Mathematical model

The linear regression model is the simplest form of regression. It assumes a linear relationship between the input features and target variable. The mathematical representation of the linear regression model is:

$$y = \beta_0 + \sum_{j=1}^{p} \beta_j x_j$$

Where p is the number of features, β are the calculated weights for each of the features, and y is the target variable.

The goal of the linear regression is to find the optimal weights β that minimize the error between the predicted values and the actual target variable. This is typically done by minimizing the ordinary least squares loss function between the predicted values and the actual target variable. The optimization problem can be formulated as:

$$\hat{\beta} = \text{argmin}_{\beta_0 \dots \beta_p} \sum_{i=1}^{n} (y^{(i)} - (\beta_0 + \sum_{j=1}^{p} \beta_j x_j^{(i)}))^2$$

3.2.2 Time complexity

There are two main approaches to solving this optimization problem: the analytical solution and iterative optimization methods (e.g., Gradient Descent (GD)). The analytical solution is given by the normal equation:

$$\hat{\beta} = (X^T X)^{-1} X^T y$$

X is an $n \cdot p$ matrix, where n is the number of instances and p is the number of features. The matrix X is augmented with a column of 1s to account for the intercept term β_0. The computational complexity of the analytical solution is dominated by matrix multiplication and inversion operations. In fact, the cost of multiplying the matrices is $O(p^2 n)$ and the cost of inverting the

pxp matrix is $O(p^3)$. Therefore, the overall computational complexity of the analytical solution is $O(p^2n + p^3)$, which becomes prohibitive for vast datasets (e.g., over 10,000 records).
The iterative methods, such as GD, calculate the gradient of the loss function with respect to the weights and update the weights iteratively until convergence. The computational complexity of the gradient calculation is $O(pn)$ per iteration, and the number of iterations required for convergence can vary depending on the learning rate and the specific dataset. However, in practice, iterative methods can be more efficient than the analytical solution for large datasets, as they do not require matrix inversion and can converge faster with appropriate hyperparameter tuning. Other methods like Stochastic Gradient Descent (SGD) can further reduce the computational cost by approximating the gradient using a subset of the data at each iteration. On the other hand, for smaller datasets, the analytical solution offers convergence in a single step, without the need for hyperparameter tuning, and guarantees a globally optimal solution.
For **inference**, the time complexity is $O(p)$ per instance, as it involves a simple dot product between the feature vector and the weight vector.

3.2.3 Spatial complexity

The spatial complexity of the linear regression model is determined by the need to store the input data, the weight vector, and any intermediate matrices used in the analytical solution. Specifically, the analytical solution requires storing the $n \cdot p$ matrix X, the $p \cdot p$ matrix X^TX, and the p-dimensional weight vector β. Therefore, the overall spatial complexity of the linear regression model is dominated by the storage of the input data and the intermediate matrices, resulting in a spatial complexity of $O(np + p^2)$.
For the GD method, the spatial complexity is reduced to $O(np + p)$, as it does not require storing the intermediate matrix X^TX.
For **inference**, the spatial complexity is $O(p)$ per instance, as it only stores the weight vector and the feature vector.

3.2.4 Internal representation

Linear regression represents the resulting weight of the features as a vector of weights, $\beta = [\beta_0, \beta_1, ..., \beta_p]$.
Specific encoding is needed for categorical features, which are typically handled through one-hot encoding, where each category is represented as a binary feature. This can lead to an increase in the number of features and potential multicollinearity issues if not handled properly.
In regard of **interpretability**, the internal representation of linear regression is straightforward and transparent, which makes it one of the most interpretable machine learning models. The weights directly indicate the strength and direction of the relationship between each feature and the target variable. This allows for a clear understanding of how each feature contributes to

the prediction, making it easier to communicate insights to stakeholders and identify important predictors in the data.

3.2.5 Data assumptions

As described by Molnar [4], linear regression relies on several key assumptions about the data to ensure model validity and the reliability of its predictions:

1. **Linear constraints:** the relationships between features and the target variable must be linear. Non-linear relationships must be manually modeled and cannot be captured automatically.
2. **Residuals normality:** the residuals $\epsilon_i = y_i - \hat{y}_i$ must follow a normal distribution. Major violations compromise statistical inference.
3. **Homoscedasticity:** residuals must have a constant variance across all levels of the features. In practice, this assumption is frequently violated. For example, in real estate, the price of very large houses is extremely variable, whereas the price of small houses is highly concentrated.
4. **Independence of measurements:** observations should not be correlated. Dependent data, such as time series, violate this assumption and should not be investigated using simple linear regression.
5. **Fixed Features:** features should be fixed and measured without error. In practice, this assumption is often violated because features can be subject to measurement errors or can change over time, leading to biased estimates of the coefficients and reduced predictive performance (attenuation bias).
6. **Absence of multicollinearity:** features should not be highly correlated. Multicollinearity causes numerical instability during the inversion of $X^T X$ and inflates the absolute value of weights. When using GD, it changes the geometric properties of the loss function, creating narrow valleys that require a significant reduction in the learning rate.

3.2.5.1 Preprocessing

The data assumptions lead to specific preprocessing steps to determine the suitability of the data for linear regression and to improve the performance of the model. These steps include:

1. **Multicollinearity identification:** calculation of the Correlation Matrix between features using the Pearson's coefficient or VIF (Variance Inflation Factor) to identify highly correlated features. $\text{VIF}_j = \frac{1}{1-R_j^2}$

Where R_j^2 is the coefficient of determination for the regression of feature j against all other features. More on R_j^2 here.

3.2.6 Predictive performance and limitations

As discussed, linear regression imposes several constraints on the data and model performance. Excellent performance, in terms of both accuracy and efficiency, is achieved when linear relationships exist between the features and the target variable.
However, in real-world scenarios, these conditions are often violated, leading to poor predictive performance. The model is particularly sensitive to outliers, which can disproportionately influence the weights and lead to skewed predictions. Additionally, linear regression is **unsuitable for capturing complex, non-linear relationships** in the data, limiting its applicability in many real-world problems where such relationships are common. Finally, the model's performance can degrade significantly when the number of features is large relative to the number of observations, leading to overfitting and poor generalization to new data. This limitation is intensified by the presence of categorical features.

3.2.7 Metrics for prediction quality

What follows is a list of the most relevant metrics for evaluating the predictive performance of linear regression models, based on the task and the data assumptions. For the general metrics see Section 3.1.2.

3.2.7.1 Feature Importance (t-statistic)

$t_{\hat{\beta}_j}$ measures the statistical significance of each coefficient, calculated as the weight divided by its standard error:

$$t_{\hat{\beta}_j} = \frac{\hat{\beta}_j}{SE(\hat{\beta}_j)}$$

Intuitively, a higher absolute value of the t-statistic indicates a more statistically significant feature.
Similarly, the higher the variance of the coefficient estimate, the less statistically significant the feature is, as the model is more uncertain about the true value of the coefficient.

3.2.7.2 p-value

p-value is a measure of the probability of *"obtaining the observed data under the null hypothesis of a statistical test"*[10]. In the context of linear regression, the null hypothesis is that the true coefficient β_j is equal to zero, meaning that the feature does not have a significant impact on the target variable. The p-value for each coefficient is calculated based on the t-statistic and indicates the probability of observing such an extreme value for $t_{\hat{\beta}_j}$ if the null hypothesis were true. A common convention is to consider a p-value less than 0.05 as statistically significant, suggesting that there is strong evidence against the null hypothesis and that the feature likely has a meaningful relationship with the target variable.

3.2.7.3 Mallows' Cp

Mallows' Cp is a model selection metric that balances model fit and complexity, calculated as:

$$Cp = \frac{SSE}{\hat{\sigma}^2} - n + 2p$$

Where $\hat{\sigma}^2$ is the estimate of the residual variance of the complete model. It is used to mitigate the problem of overfitting by penalizing the addition of unnecessary features.

3.2.8 Diagnostic plots

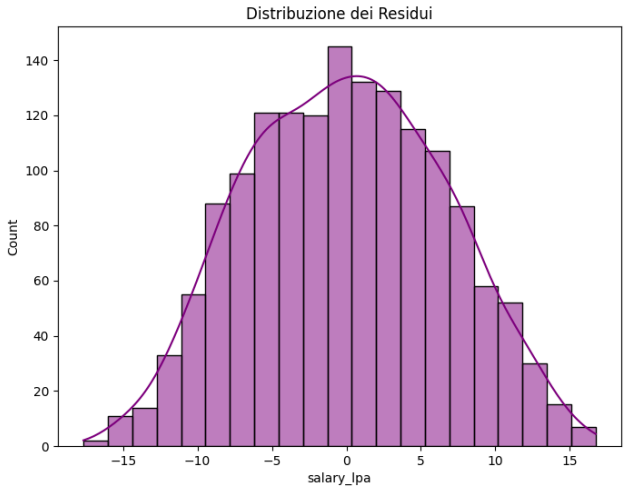
The use of diagnostic plots is crucial to visually assess the assumptions of linear regression and to identify potential issues with the model fit.

3.2.8.1 Actual vs Predicted

Scatter plot with actual values y_i on the y-axis and predicted values $\hat{y}_i$ on the x-axis. If the model fits well, the points should be concentrated around the diagonal line $y = \hat{y}$. Deviations from this pattern can indicate various issues with the model fit.

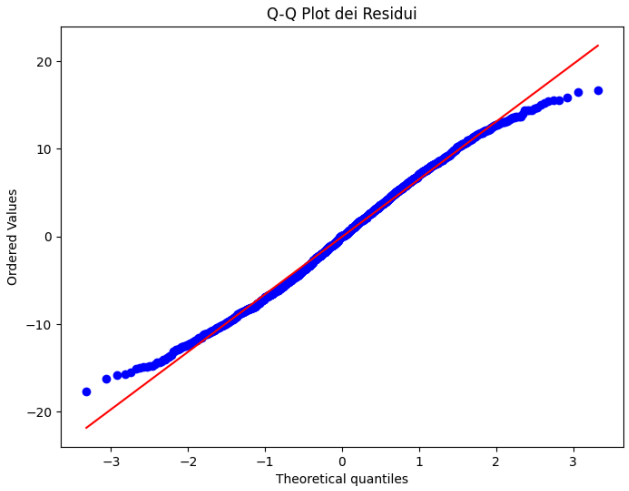
3.2.8.2 Histogram of residuals distribution

Distribution of the residuals $\epsilon_i = y_i - \hat{y}_i$. It is especially useful to identify violations of the normality assumption. A normal distribution of residuals is expected for valid inference, and deviations from this pattern can indicate issues with the model fit or the presence of outliers.



Histogram of residuals distribution example for linear regression.

3.2.8.3 Q-Q Plot (Quantile-Quantile)



Q-Q Plot of residuals example for linear regression.

Another way to check the normality of residuals is through a Q-Q plot, which compares the quantiles of the residuals to the quantiles of a normal distribution. If the residuals are normally distributed, the points in the Q-Q plot should approximately follow a straight line. Deviations from this line, especially at the tails, can indicate non-normality of the residuals, which may affect the validity of statistical inference based on the model.

3.2.8.4 Residuals vs Fitted Values

Scatter plot with fitted values $\hat{y}_i$ on the x-axis and residuals $\epsilon_i = y_i - \hat{y}_i$ on the y-axis.
If the model fits well, the points should be randomly scattered around the horizontal line $\epsilon = 0$ without displaying any systematic pattern. This plot is used to identify violations of regression assumptions (Section 3.2.5). For example, increasing dispersion (forming a funnel shape) shows heteroscedasticity, while systematic patterns (e.g., a curve) indicate non-linearity that the model cannot capture.

3.2.9 Explainability and interpretability metrics

3.2.9.1 Feature Effect

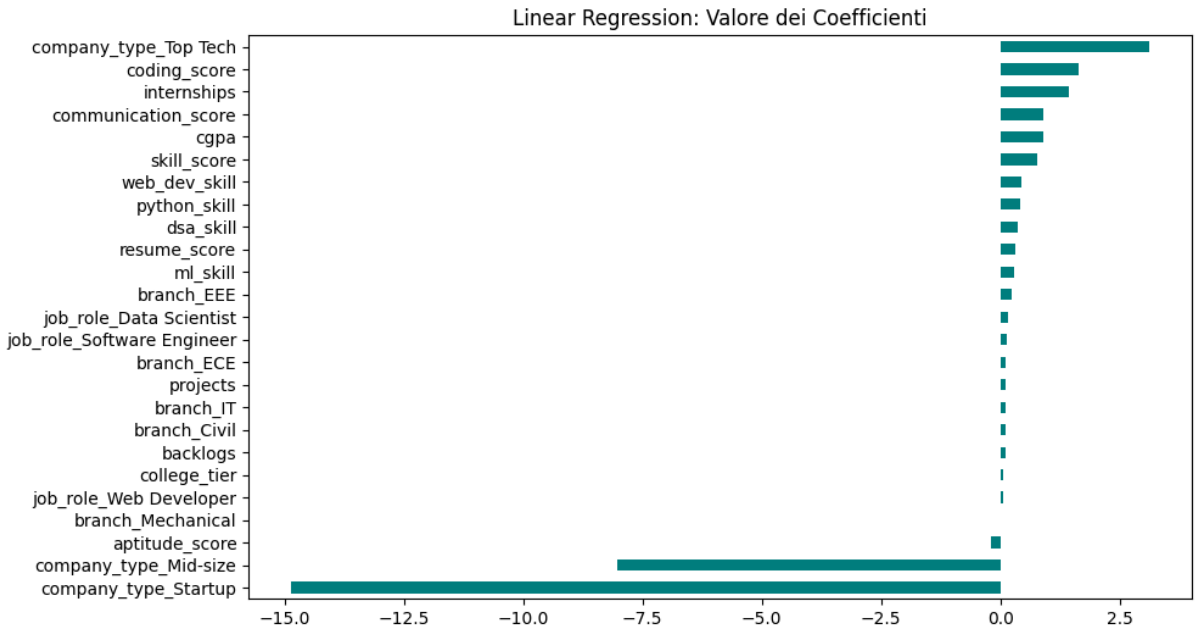
Linear regression naturally provides a direct measure of the feature effect on the target variable through the coefficients β_j. The effect of a feature x_j on the prediction for an instance i can be calculated as:

$$\text{effect}_j^{(i)} = \beta_j x_j^{(i)}$$

The standard visualization shows a box plot of the calculated effect in the 25th, 50th, and 75th percentiles (quantiles) for each feature. This enables users to quickly identify the features with the most significant effect on the predictions, as well as the distribution of the effects across the dataset.
This plot is not useful if the data are normalized, as the effect is calculated as the product of the coefficient and the feature value, and normalization can obscure the true impact of the features on the predictions.

3.2.9.2 Weight Plot

For a direct visualization of the importance of each feature, a weight plot can be used, which shows the coefficients β_j for each feature. The coefficients indicate the strength and direction of the relationship between each feature and the target variable.



Weight Plot of coefficients example for linear regression.

3.2.10 Interpretability and explainability limitations

As discussed, linear regression is one of the most interpretable machine learning models due to its transparent internal representation and the direct relationship between features and predictions. The impact of each feature on the prediction can be easily understood through the coefficients, which indicate how much the prediction changes with a one-unit change in the feature, holding all other features constant.

What makes the model highly interpretable also limits its predictive capacity; the assumption of linearity is as restrictive as it is understandable.

3.3 Logistic regression

3.3.1 Mathematical model

Logistic regression is a model primarily used for binary classification problems where the response variable is dichotomous. Similar to linear regression(Section 3.2) it uses a linear combination of the input features, but instead of directly outputting a continuous value, it applies a logistic function to map the output to a probability between 0 and 1.

$$\sigma(z) = \frac{1}{1 + \exp(-z)}$$

The output can consequently be interpreted as the probability that the input instance belongs to the positive class (y = 1). In particular, the probability that an instance belongs to the positive class is given by:

$$P(y^{(i)} = 1 \mid x^{(i)}) = \frac{1}{1 + \exp(-(\beta_0 + \sum_{j=1}^{p} \beta_j x_j^{(i)}))}$$

And the probability of the negative class follows as $P(y^{(i)} = 0) = 1 - P(y^{(i)} = 1)$, consistent with the Bernoulli distribution.

The goal of the logistic regression is to find the parameters β that best fit the data, which is done by maximizing the **likelihood** of observing the data given the parameters.

$$L(\beta; y, X) = \prod_{i=1}^{n} P(y^{(i)} = 1)^{y^{(i)}} \cdot (1 - P(y^{(i)} = 1))^{1-y^{(i)}}$$

For numerical stability **log-likelihood** is usually calculated:

$$\ell(\beta) = \sum_{i=1}^{n} \left[y^{(i)} \log(P(y^{(i)} = 1)) + (1 - y^{(i)}) \log(1 - P(y^{(i)} = 1)) \right]$$

Differently from linear regression, the logistic regression does not have a closed-form solution for the parameters that maximize the likelihood so only iterative optimization algorithms like GD are available

3.3.2 Time complexity

As described in Section 3.2.2, the time complexity of GD is $O(pn)$ per iteration, and the number of iterations required for convergence can vary depending on the learning rate and the specific dataset. For **inference** the time complexity is again, $O(p)$ per instance.

Since the optimization is iterative, the overall time complexity can be less predictable than linear regression, especially if the data has features that lead to slow convergence (e.g., highly correlated features or features that cause the decision boundary to be close to the data points). However, in practice, logistic regression is often computationally efficient for moderate-sized datasets and can be scaled to larger datasets using SGD or mini-batch gradient descent.

3.3.3 Spatial complexity

The model stores a vector of coefficients β of size p, which requires $O(p)$ memory. During training, additional memory is needed to store the intermediate values for the optimization algorithm, which can be $O(n)$ for batch gradient descent due to the need to compute the predictions for all instances in each iteration.

For inference, the memory complexity is $O(p)$ for storing the coefficients and $O(1)$ for the input instance, leading to an overall spatial complexity of $O(p)$.

3.3.4 Internal representation

The model internally represents the values as a vector of weights, $\beta = [\beta_0, \beta_1, ..., \beta_p]$. The presence of categorical features is standardly resolved using one-hot encoding, just like in linear regression.

For **interpretability**, the model remains transparent since the coefficients still indicate the strength and direction of the relationship between each feature and the target variable, but the interpretation is less intuitive than in linear regression due to the non-linear transformation applied to the output by the logistic function. An increase in one feature in logistic regression leads to a multiplicative change in the odds of the positive class, rather than an additive change in the predicted value.

3.3.5 Data assumptions

Before discussing the data assumptions, we need to clarify the role of the odds. The odds of an event are defined as the ratio of the probability of the event occurring to the probability of it not occurring:

$$\text{odds} = \frac{P(y=1)}{P(y=0)} = \exp\left(\beta_0 + \sum_{j=1}^{p} \beta_j x_j\right)$$

This formulation more easily shows how the coefficients β_j affect the predictions.

$$\frac{\text{odds}_{x_j+1}}{\text{odds}} = \exp(\beta_j)$$

Just like in linear regression, the idea of maintaining the interpretability of the model through a linear combination of the features leads to several assumptions regarding the data and model performance:

1. **Linearity of the log-odds:** $\log(\text{odds}) = \beta_0 + \sum_j \beta_j x_j$. The relationship is linear in the **log-odds space**, not in the probability space. Non-linear relationships remain uncaptured and must be added manually.
2. **Complete separation:** if a feature perfectly separates the two classes, the model cannot identify a finite weight since the algorithm will tend to push the weight to $+\infty$ or $-\infty$, diverging. Usually, the solution is to remove this separating feature, since it is likely just a proxy for the target variable.
3. **Binomial distribution:** the response variable must follow a **Bernoulli distribution**.
4. **Independence of measurements:** observations should not be correlated. This means that the model is not suitable for temporal data without proper control, as well as for data with strong dependencies between observations.
5. **Fixed features:** features must be considered constants, without measurement error.

6. **Absence of multicollinearity:** highly correlated features cause coefficient instability. This is particularly relevant because iterative methods may oscillate and require very small learning rates (α).

Homoscedasticity is no longer a requirement because the response can only be 0 or 1.

3.3.5.1 Preprocessing

To verify the suitability of logistic regression in the classification task for a given dataset, several methods can be used. The correlation matrix can be used to identify highly correlated features, which can lead to multicollinearity issues. If such features are found, they can be removed or combined to reduce redundancy and improve model stability.
For identifying anomalies in the other assumptions, plot visualization can be used. See Section 3.3.8 for more details.

3.3.6 Predictive performance and limitations

The imposed assumptions of logistic regression can lead to good performance whenever these assumptions are met. The probabilistic interpretation gives insight into the confidence of the prediction, which is a significant advantage in many applications. Computationally, the model achieves fast training thanks to the iterative methods used to solve the loss function.
However, when the relationships are more complex than basic linear combinations, the predictions become less accurate. In the context of imbalanced classes, the training process can be dominated by the majority class, leading to poor performance on the minority class. Additionally, logistic regression, like linear regression, is highly influenced by outliers, leading to very unstable predictions. Finally, the limitation of complete separation can prevent the model from converging to a solution.

3.3.7 Metrics for prediction quality

The following is a list of metrics for evaluating the predictive performance of logistic regression in the context of the classification task. For the general metrics see Section 3.1.1.

3.3.7.1 Z-Statistic and p-value

The equivalent of the t-statistic for logistic regression is the Z-statistic, which measures the statistical significance of each coefficient in the model. It is calculated as:

$$Z = \frac{\beta_j}{\text{SE}\ (\beta_j)}$$

Where SE (β_j) is the standard error of the coefficient β_j. A coefficient with a large absolute Z-value indicates that the coefficient is significantly different from zero, suggesting that the corresponding feature has a significant impact on the predicted probabilities.
The p-value associated with the Z-statistic represents the probability of observing such an

extreme Z value under the null hypothesis that $\beta_j = 0$. For more details on the p-value, see Section 3.2.7.2.

3.3.8 Explainability and interpretability metrics

Using plots to visualize the data and the model's predictions can help identify potential violations of the assumptions of logistic regression and provide insights into the model's performance.

3.3.8.1 Feature Effect

Similar to Linear Regression (Section 3.2.9.1), the feature effect plot represents the impact of a feature on the prediction. In the logistic regression case, the effect is not on the predicted value but on the predicted probability, which is more intuitive to understand for most users.

For every feature value, calculate:

$$P(y = 1 \mid x_j = v, x_{\text{other}} = \text{mean})$$

3.3.8.2 Weight Plot

Likewise to linear regression, the weight plot shows the coefficients of the features. However, in logistic regression, the coefficients do not directly correspond to changes in the predicted probability, but rather to changes in the log-odds of the positive class.

3.3.8.3 Odds Ratio

Direct expression of the feature impact on the odds:

$$\text{OR}_j = \exp(\beta_j)$$

It represents the multiplicative change in the odds of the positive class for a one-unit increase in the feature x_j, holding all other features constant.

3.3.9 Interpretability and explainability limitations

The logistic regression, while being more interpretable than many other machine learning models, has some limitations in terms of interpretability. Firstly, the interpretation of the coefficients is less intuitive than in linear regression due to the non-linear transformation applied to the output by the logistic function. An increase in one feature in logistic regression leads to a multiplicative change in the odds of the positive class, rather than an additive change in the predicted value.

This multiplicative effect is even less straightforward if we consider that the effect of a feature on the predicted probability depends on the values of all the other features, making it difficult to provide a simple explanation of the effect of a single feature without considering the context of the other features.

3.4 Support Vector Machine (SVM)

3.4.1 Mathematical model

A Support Vector Machine is a classifier that finds the optimal hyperplane that separates two classes by maximizing the distance (margin) between the hyperplane and the closest points of each class.

In the case of bidimensional data, the hyperplane is a line; for three-dimensional data, it is a plane; for p-dimensional data, it is a hyperplane defined by:

$$\beta_0 + \beta_1 x_1 + \beta_2 x_2 + ... + \beta_p x_p = 0$$

Where $\beta = [\beta_1, ..., \beta_p]$ is the normal vector to the hyperplane, and β_0 is the intercept.

3.4.1.1 Hard-Margin SVM (Linearly Separable Data)

In the ideal case in which the data are perfectly and **completely separable**, the goal is to find the coefficients $\beta_0, \beta_1, ..., \beta_p$ that maximize the margin. The intuition, is that a hyperplane with a large margin is more **robust** to variations in the data and generalizes better to unseen data. The separation condition is then the following:

$$y_i(\beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + ... + \beta_p x_{ip}) \geq 1 \quad \forall i$$

Where $y_i \in \{-1, +1\}$ is the class label.

The distance between a point in the training set and the hyperplane is given by:

$$r_i = \frac{y_i(\beta_0 + \sum_{j=1}^{p} \beta_j x_{ij})}{|| \beta ||}$$

Where $|| \beta || = \sqrt{\sum_{j=1}^{p} \beta_j^2}$ is the **euclidean norm** of the coefficient vector.

To maximize the margin, defined as the distance between the hyperplane and the closest points, we can express it as:

$$M = \frac{1}{|| \beta ||}$$

is equivalent to minimizing $|| \beta ||$. This formulation leads to the following optimization problem:

$$\min_{\beta, \beta_0} \frac{1}{2} || \beta ||^2$$

Under the constraint that:

$$y_i \left(\beta_0 + \sum_{j=1}^{p} \beta_j x_{ij} \right) \geq 1 \quad \forall i = 1, ..., n$$

3.4.1.2 Soft-Margin SVM (Non Linearly Separable Data)

In the realistic case where the data are **not linearly separable**, due to the presence of noise, outliers, or overlapping classes, we allow some points to violate the margin. This can be achieved by introducing **slack variables** $\xi_i \geq 0$ that measure how much a point violates the margin:

$$y_i \left(\beta_0 + \sum_{j=1}^{p} \beta_j x_{ij} \right) \geq 1 - \xi_i \quad \forall i$$

Once we allow violations, we need to penalize them in the objective function to prevent the model from simply classifying all points as the majority class. This leads to the following optimization problem:

$$\min_{\beta, \beta_0, \xi} \left[\frac{1}{2} \mid\mid \beta \mid\mid^2 + C \sum_{i=1}^{n} \xi_i \right]$$

The objective function shows the importance of two components of the model. First, the margin of the hyperplane (first term) and second, the violations of the margin (second term). The parameter C is a **regularization hyperparameter** that controls the trade-off between maximizing the margin and minimizing violations. A high value of C will prioritize minimizing violations, approaching the hard-margin SVM and potentially leading to overfitting. A low value of C will prioritize maximizing the margin, allowing more violations.

3.4.1.3 Kernel SVM

The main limitation of linear SVM is that it only works if the data are approximately linearly separable. For data with non-linear patterns, SVM uses the **kernel trick**. The idea is to transform the data into a higher-dimensional space where the initially non linearly-separable data become linearly separable.

Computing the transformation explicitly is computationally expensive. The **kernel trick** allows us to do this implicitly.

Instead of calculating the transformation $\phi(x_i) = [f_1(x_i), f_2(x_i), ..., f_m(x_i)]$ and then calculating the scalar product $\phi(x_i)^T \phi(x_j)$, using a kernel function that returns the same result directly from the original data:

$$K(x_i, x_j) = \phi(x_i)^T \phi(x_j)$$

without the need to explicitly calculate ϕ.

The most common kernels are [11]:

1. Linear Kernel

$$K(x_i, x_j) = x_i^T x_j$$

This is the default kernel and corresponds to the original linear SVM. It does not perform any transformation and is suitable for linearly separable data.

2. Polynomial Kernel

$$K(x_i, x_j) = (x_i^T x_j + 1)^d$$

Transforms the data into a space of polynomials of degree d. Useful for polynomial patterns and especially precise in describing curved boundaries.

3. RBF Kernel (Radial Basis Function - Gaussian)

$$K(x_i, x_j) = \exp(-\gamma \mid\mid x_i - x_j \mid\mid^2)$$

Transforms the data into an infinite-dimensional space. It is the **most commonly used kernel** thanks to its flexibility, which makes it suitable for describing complex patterns. It also has only a single hyperparameter γ (γ) to tune, making it easy to use.
This hyperparameter controls the influence of a single training example. The larger the value of γ, the closer other examples must be to be affected, leading to a higher risk of overfitting. Conversely, a smaller value of γ means that even points far from the decision boundary can influence it. In this case, a value of γ that is too small can lead to underfitting, as the model may fail to capture the complexity of the data.

4. Sigmoid Kernel

$$K(x_i, x_j) = \tanh(\alpha x_i^T x_j + c)$$

The sigmoid kernel resembles the activation function of a neural network and can be used in datasets where the relationship between features is expected to be "threshold-like" [12], split into hard decision regions. However, it is less commonly used than the RBF kernel and can be more difficult to tune, but it can sometimes be useful as an alternative to a full neural network.

3.4.2 Time complexity

The time complexity for SVM training is heavily dependent on the different variants of the algorithm and the size of the dataset. In general, the training time complexity is between $O(n^2 p)$ and $O(n^3 p)$, where n is the number of training samples and p is the number of features. This is because SVM training involves solving a quadratic optimization problem, which can be computationally expensive, especially for large datasets. To mitigate the computational bottleneck of kernel use in SVM, various optimizations can be employed. Approximate kernel methods, such as the Nyström method or Random Fourier Features (RFF), can reduce the computational cost of kernel SVMs by approximating the kernel matrix using sampling or random Fourier transformations. Additionally, Sequential Minimal Optimization (SMO) breaks down the optimization problem into smaller subproblems that can be solved analytically. Even with these efficient optimization algorithms, using sophisticated kernels leads to higher computational costs, which is the trade-off for better predictive performance.
On the other hand, linear SVMs with proper optimization, like Linear Programming (LP) or SGD, lower their time complexity to $O(np)$. This makes linear SVMs more scalable for large

datasets, though they are limited to linear patterns.

For **inference**, the time complexity is again related to the kernel function and the number of support vectors, which usually grows with the size of the training set. Generally, it is between $O(mp)$ and $O(mn)$, where m is the number of support vectors, while for linear SVMs the inference time complexity is $O(p)$, as the prediction is a simple dot product between the input features and the coefficients of the hyperplane.

3.4.3 Spatial complexity

For the memory complexity, the main bottleneck is the storage of the kernel matrix, which has a size of $O(n^2)$, making it impractical for large datasets. For linear SVMs, the memory complexity is much lower, at $O(p)$, as it only needs to store the coefficients of the hyperplane. The number of support vectors also affects the memory complexity, as each support vector requires storing its corresponding coefficient and features. In general, the memory complexity can be between $O(n^2 + mp)$ and $O(mp)$, where m is the number of support vectors.

To mitigate the memory issues that for large datasets mean impracticality, the same approximations used for time complexity can be applied, reducing the complexity of the problem in more limited and manageable subproblems.

3.4.4 Internal representation

The model represents the solution as a **linear combination of kernel products** of the form

$$f(x) = \beta_0 + \sum_{i=1}^{n} \alpha_i y_i K(x, x_i)$$

Where:

- α_i are the dual coefficients (not directly the β_j)
- Only a fraction of the training points have $\alpha_i > 0$, these are the support vectors

For **interpretability**, this representation leads to a series of disadvantages. First, it is not immediately clear why a particular point is a support vector, as it depends on the complex interactions of the data and the kernel. Second, for non-linear kernels, the transformation ϕ is implicit and not visualizable, making it difficult to understand how the model is making decisions. Finally, the interpretation of α_i is **not intuitive**, as it does not directly correspond to feature importance but rather to the influence of support vectors on the decision boundary. So, while SVM can be powerful for prediction, its internal representation poses significant challenges for interpretability. This is especially true for non-linear kernels, giving less insight into the decision-making process compared to more interpretable models like linear regression or decision trees.

3.4.5 Data assumptions

As discussed, the different variants of SVM have different characteristics, which extends to data assumptions too.

For the linear SVM, as the name suggests, the main assumption is that the data are approximately linearly separable by a hyperplane. If this assumption is not met, predictive performance decays significantly, and the model may not be able to capture the underlying patterns in the data.

Some general assumptions do exist for both linear and kernel SVMs. These include:

1. **Class balance:** SVM can be sensitive to imbalanced classes, as it focuses on maximizing the margin between classes. If one class is significantly more frequent than the other, the decision boundary may be biased towards the majority class.
2. **Feature scaling:** SVM is sensitive to the scale of the features, as it relies on the distance between data points. If features are on different scales, the model may give more weight to those with larger ranges. Therefore, it is important to standardize or normalize the features before training an SVM.

No other structural assumption emerges from the mathematical formulation giving the SVM a certain flexibility in modeling different types of data, as long as the kernel is chosen appropriately to capture the underlying patterns.

3.4.6 Predictive performance and limitations

The SVM is a powerful and versatile algorithm that is able to achieve high predictive performance, especially when the data have non-linear patterns. The Soft-margin SVM decrese the impact of outliers and a higher number of features are an improvement over the linear models, which can easily overfit in these scenarios.

However, in real-world application, with massive amount of data, the computational cost of training and inference for the kernel SVM can become prohibitive. The ideal scenario of use is consequently for medium-sized datasets (up to tens of thousands of samples) with complex patterns that require non-linear modeling. For larger datasets, linear SVMs can be used, but they are limited to linear patterns and may not capture the complexity of the data, leading to lower predictive performance. Additionally, SVMs can be sensitive to the choice of hyperparameters which can further affect their performance. Finally, there is no direct probabilistic interpretation of the SVM outputs such as in logistic regression.

Therefore, while SVMs can be powerful for prediction, they may not always be the best choice for every problem, especially when scalability and interpretability are important considerations.

3.4.7 Metrics for prediction quality

The following is a list of SVM specific metrics for evaluating the predictive performance of the model. For the general metrics see Section 3.1.1.

3.4.7.1 Distance from hyperplane

This represents the most direct measurement of the confidence of the SVM prediction. The distance of a point x from the hyperplane is given by:

$$d(x) = \frac{\mid \beta_0 + \sum_{j=1}^{p} \beta_j x_j \mid}{\|\beta\|}$$

Where $\|\beta\| = \sqrt{\sum_{j=1}^{p} \beta_j^2}$ is the Euclidean norm of the coefficient vector. The closer the point is to the hyperplane (i.e., $d(x)$ close to 0), the less confident the prediction is, while points far from the hyperplane (i.e., $d(x)$ with large value) are predicted with higher confidence.

3.4.8 Explainability and interpretability metrics

3.4.8.1 Platt Scaling

Out of the box SVM does not produce a measure of the uncertainty of its predictions, as it outputs a hard classification ($+1$ or -1) rather than a probability. To obtain estimates of the probability $P(y = 1 \mid x)$, a common approach is to use **Platt scaling**, which fits a logistic regression model to the SVM decision values $f(x) = \beta_0 + \sum_{j=1}^{p} \beta_j x_j$ (which are proportional to the signed distances from the hyperplane) on a validation set. The formula for Platt scaling is:

$$P(y = 1 \mid x) = \frac{1}{1 + \exp(Af(x) + B)}$$

Where A, B are parameters learned on a validation set, calibrating the decision values to probabilities. Although Platt scaling can provide a way to obtain probabilistic outputs from SVMs, it is important to note that the probabilities obtained through this method may not be well-calibrated, especially if the validation set used for calibration is not representative of the test set. Therefore, while Platt scaling can be useful for certain applications, it should be used with caution and its outputs should be interpreted carefully.

3.4.8.2 Support vectors and their weights (α_i)

Analysing the support vectors and their corresponding coefficients α_i can provide insights into the decision-making process of the SVM. Support vectors are the data points that lie closest to the decision boundary and have a direct influence on its position.

$$f(x) = \beta_0 + \sum_{i=1}^{n} \alpha_i y_i K(x, x_i)$$

The weights α_i associated with these support vectors indicate their importance in determining the decision boundary, the higher the value of α_i, the more influence the corresponding support vector has on the decision boundary. In kernel based SVMs, their interpretation is less intuitive.

3.4.8.3 Feature importance

To assess the importance of individual features in the SVM model, we can use techniques that are model-agnostic, as SHAP values, since the SVM does not provide feature importance scores directly. The main idea is to measure how the model's predictions change when we perturb or remove a feature, which can give us insights into the importance of that feature for the model's decision-making process.

3.4.9 Interpretability and explainability limitations

As discussed in Section 3.4.4, SVM as a model has very important limitations in interpretability. Linear SVM can be explained through the coefficients, which indicate the orientation of the decision boundary.
Additionally, support vectors can be identified and analyzed, which provides some insights into the model's decision-making process.
Therefore, the model's behavior is still partially explainable, but the lack of transparency in the mathematical representation makes it less interpretable than simpler models. These limitations are particularly pronounced when using non-linear kernels, which transform the data into a higher-dimensional space where the decision boundary is not easily visualizable. The interpretation of the weights α_i of the support vectors is **not straightforward**, as they do not directly correspond to feature importance but rather to the influence of the support vectors on the decision boundary.\ Additionally, there is no natural way to assess feature importance directly nor to trace the reasoning process of the model, as the decision boundary is determined by a complex combination of support vectors and their corresponding weights, which can be difficult to communicate and understand, especially for non-experts.
Therefore, while SVM can be powerful for prediction, its interpretability limitations should be carefully considered when choosing it for a particular application, especially when interpretability is a key requirement.

3.5 Decision Tree

3.5.1 Mathematical model

A decision tree is a model that recursively splits the data space into rectangular regions, creating a **hierarchical structure of decision nodes**. The resulting tree is built using a greedy top-down approach, where at each node the best feature and threshold are chosen to maximize the purity of the resulting child nodes (for classification) or minimize the variance (for regression). The final predictions are made by traversing the tree from the root to a leaf, following the decision rules at each node. The leaf reached determines the predicted class (classification) or the predicted value (regression), while the internal nodes represent the decision rules based on feature thresholds.

3.5.1.1 Splitting criteria

A multitude of splitting criteria exist; the following are the most common (for a more complete list, see [13], [14]). These criteria are usually based on measures of impurity, which quantify how mixed the classes are in a node. The goal is to find splits that create child nodes that are purer than the parent node.

1. **Gini Index (Classification):**

Measures based on **impurity** for classification problems:

$$\text{Gini } (D) = 1 - \sum_{i=1}^{K} p_i^2$$

Where p_i is the proportion of instances belonging to class i in the node, and K is the total number of classes. A Gini index of 0 means that all instances in the node belong to the same class (pure node). A Gini index equal to $1 - \frac{1}{K}$ means that the instances are uniformly distributed among the classes (completely impure node). For binary classification ($K = 2$), the maximum Gini index is 0.5 when the classes are perfectly balanced.

To measure the **Gini Gain** of a split, we calculate the weighted average reduction in Gini impurity:

$$\text{IG} = \text{Gini (parent)} - \sum_{\text{child}} \frac{\mid D_{\text{child}} \mid}{\mid D \mid} \text{Gini (child)}$$

2. **Entropy (Classification):**

Another measure of impurity for classification problems, based on the concept of entropy from information theory:

$$\text{Entropy } (D) = -\sum_{i=1}^{K} p_i \log_2(p_i)$$

The closer the entropy is to 0, the purer the node. The maximum entropy occurs when the classes are perfectly balanced, which is $\log_2(K)$ for K classes. For binary classification ($K = 2$), the maximum entropy is 1 when the classes are perfectly balanced.

From the entropy we can derive the **Information Gain** of a split, which measures how much the entropy is reduced by the split similarly to Gini Gain:

$$\text{IG} = \text{Entropy (parent)} - \sum_{\text{child}} \frac{\mid D_{\text{child}} \mid}{\mid D \mid} \text{Entropy (child)}$$

The Gini and Entropy measures often lead to similar splits, but Gini is computationally faster, which is why it is more commonly used in practice.

3.5.1.2 Mean Squared Error (MSE) (Regression):

Measures based on **variance** for regression problems. The goal is to find splits that minimize the variance of the target variable in the child nodes. The MSE of a node is calculated as:

$$\text{MSE } (D) = \frac{1}{\mid D \mid} \sum_{i=1}^{\mid D \mid} (y_i - \bar{y})^2$$

Where $\bar{y} = \frac{1}{\mid D \mid} \sum_{i=1}^{\mid D \mid} y_i$ is the mean of the target variable in the node.

3.5.2 Time complexity

The time complexity of the decision tree is bounded by the process of evaluating the best split at each node, which involves scanning through the data and calculating the splitting criterion for each feature.

$$O(n^2 \cdot p \cdot \log(n))$$

Where n is the number of observations and p the number of features.
In fact, sorting the data for each feature takes $O(n \log(n))$, and this process is repeated for each of the p features at each level of the tree. If the tree is grown to every leaf (no pruning), the number of nodes is on the order of $O(n)$, giving the final complexity. The quadratic term in the worst case can represent a bottleneck for large datasets, especially when the tree is allowed to grow deep and capture noise in the data. In this case, gradient-based tree algorithms (Section 3.7) could be preferred, not only for their better time complexity but also for their improved predictive performance and regularization capabilities.
On the other hand, as described in the official Scikit-learn documentation [15], the time complexity can be optimized to $O(n \cdot p \cdot \log(n))$ thanks to clever tracking of the general order of indices of the features, avoiding sorting at each node. Feature-level parallelization can be achieved by evaluating the splits to additionally improve performance.
For **inference**, the time complexity is dependent on the depth of the tree, which in the worst case can be $O(n)$ (degenerate tree) and in the best case $O(\log(n))$ (balanced tree).

3.5.3 Spatial complexity

The spatial complexity is determined by the need to store the tree structure and the metrics for the evaluation of the splits. The spatial complexity is consequently $O(n \cdot p)$ during training but drops to $O(n)$ during inference as we only need to store the tree structure and the thresholds for the splits, which is proportional to the number of nodes in the tree.

3.5.4 Internal representation

A decision tree is represented internally as a recursive tree structure, as suggested by the name. Each node:

```
Node(feature=x1, threshold=5.5, left=Node(...), right=Node(...))
```

contains the feature used for splitting, the threshold value for the split and the reference to the left and right child nodes while a leaf only contains the predicted class or value and the number

of samples in that leaf.
This structure not only allows for efficient traversal during inference but, for **interpretability**, it provides a clear and transparent representation of the decision-making process. Each path from the root to a leaf corresponds to a specific set of conditions on the features.
Still, with deep trees, understanding the global structure can become difficult, even if the local decision paths remain interpretable.
An advantage of the decision tree structure is the natural handling of categorical features without the need for encoding.

3.5.5 Data assumptions

The only assumption of decision trees is that the data can be split based on feature thresholds to create **homogeneous** groups. Imbalanced classes can affect predictive performance regarding the minority class, leading to a very deep tree for the majority class.
This is a very weak assumption compared to linear models, which require linearity, normality, homoscedasticity, and independence of features. However, it is not always possible to satisfy it, and sometimes resampling or proportional weighting is needed to address it. Similarly to linear models, decision trees can be affected by multicollinearity, as they tend to prefer one feature over another when they are highly correlated, which can lead to instability in the tree structure and potentially affect the interpretability of the model.

3.5.6 Predictive performance and limitations

Decision trees are powerful models in the context of classification, especially when the relationship between features and target is complex and non-linear, for example, in a context with multiple categorical features. Feature interactions are naturally captured by the tree structure without explicitly modeling them. As said previously, decision trees can handle categorical features without the need for encoding, which can be a significant advantage in many real-world datasets. Moreover, they do not require any assumptions about the distribution of the data or the linearity of relationships between features and target variable, making them versatile for a wide range of problems.
All these advantages lead to a model that can capture automatically complex patterns in a vast variety of datasets.
On the contrary, decision trees can be inaccurate on purely linear data, where a simple linear model would achieve better performance. In general, decision tree regressors tend to be inaccurate as they approximate the target variable with piecewise constant predictions, which can lead to high bias.
Decision trees are even prone to overfitting, especially when the tree is allowed to grow deep and capture noise in the training data and in high dimensionalities. This can lead to poor generalization performance on unseen data.

In addition, they can also be biased towards features with more levels, which can lead to misleading interpretations of feature importance.
Finally, decision trees can be unstable, meaning that small changes in the training data can lead to significantly different tree structures and predictions. This is because the tree-building process is greedy and makes locally optimal decisions at each node, which can be sensitive to variations in the data.

3.5.7 Metrics for prediction quality

To evaluate the predictive performance of decision trees, we can use both general metrics for classification and regression, as well as specific metrics that take advantage of the tree structure. For the common metrics, see Section 3.1.1 (Classification) and Section 3.1.2 (Regression).

3.5.7.1 Confidence score or Probability on the leaf

To estimate the confidence of a prediction, we can use the distribution of classes in the leaf node reached by the instance. The confidence score for a predicted class can be calculated as the proportion of instances of that class in the leaf compared to the total number of instances in that leaf:

$$\text{Confidence} = \frac{\#\text{ instances of the predicted class in the leaf}}{\#\text{ total instances in the leaf}}$$

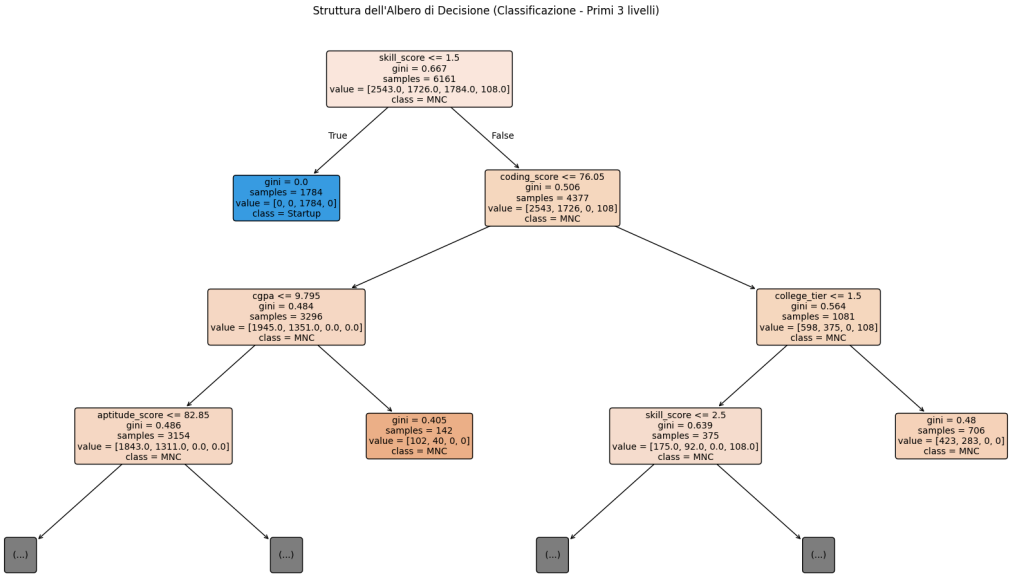
This metric could be used to filter predictions based on the level of confidence, for example by accepting only predictions with a confidence score above a certain threshold (e.g., 0.8). This can be particularly useful in applications where the cost of false positives or false negatives is high, allowing us to focus on predictions that the model is more certain about.

3.5.8 Explainability and interpretability metrics

Visualizations, such as plots and feature importance, can improve the interpretability of decision trees. The following plots and metrics are specifically chosen to extrapolate insights from the decision trees and to understand the decision-making process of the model, rather than just evaluating its predictive performance.

3.5.8.1 Tree visualization

The first and most intuitive way to understand a decision tree is to visualize it. The nodes show the features and their thresholds, while the leaves show the predicted classes. In this way, the decision path is easily traceable, and the most important splitting features are clear.



Visualization of a decision tree.

For deep trees, the complete visualization can become cluttered and difficult to interpret. In such cases, it can still be useful to visualize only the top levels of the tree, which capture the most important splits.

3.5.8.2 Feature importance

Measures how often a feature is used as a splitting criterion and how much it reduces the average impurity:

$$\text{Importance}_j = \frac{\sum_{\text{nodes with feature } j} \text{IG}_{\text{node}} \times \frac{n_{\text{node}}}{n}}{\text{total reduction of impurity}}$$

The more a feature is used for splitting and the more it reduces impurity, the higher its importance score. This metric helps to identify not only which features are most influential in the model, but also to understand the relative importance of different features in the decision-making process of the tree. Indeed, the most dividing features appear at the top and can be identified easily, but the feature importance metric also allows the identification of less important features used for splitting in deeper levels of the tree.

3.5.8.3 Path to prediction

For a single instance, it is possible to trace the complete path from root to leaf and list all the comparisons (feature <= threshold) that led to the prediction. This is a **huge advantage for explainability** compared to black-box models. Every prediction is completely traceable locally.

> Instance: (x1=3.2, x2=1.5, x3=A)

```
Path to prediction:
- Node 1: x1 <= 5.5 (True)
- Node 2: x2 <= 2.0 (True)
- Node 3: x3 == B (False)
- Node 4: x3 == A (True)

Prediction: Class 1 (Confidence: 0.8)
```

This is especially useful in contexts where understanding the reasoning behind a specific prediction is crucial, such as in medical diagnosis or credit scoring. By examining the path to prediction, we can identify which features and thresholds were most influential in the decision-making process for that particular instance, providing insights into the model's behavior and potentially uncovering any biases or issues in the data.

3.5.9 Interpretability and explainability limitations

Decision trees are generally considered interpretable models, but they have some limitations in terms of interpretability too.

One of the main limitations is that as the tree grows deeper and more **complex**, it can become difficult to understand the global structure of the model and how different features interact with each other across the entire tree. While it is possible to trace the decision path for a single instance, understanding the overall reasoning process of the model can be challenging when there are many nodes and interactions between features.

In the context of correlated features, decision trees can mask the importance of one feature in favor of another, losing valuable insights about the data. Finally, for regression tasks, piecewise-constant predictions can make it difficult to understand the relationship between features and the target variable, especially when the tree is deep and captures complex interactions.

Even with these limitations, decision trees still remain "white boxes" that are easy to understand.

3.6 Random forest

3.6.1 Mathematical model

Random forest is an ensemble method that combines multiple decision trees to improve prediction performance compared to a single tree. In particular, the idea is to address the overfitting and instability problems discussed in Section 3.5.6. There exist multiple ways to achieve this. The most common are:

1. **Bootstrap Aggregating (Bagging):** training every tree on a different sample of the dataset.
2. **Feature Randomness:** every split considers only a random subset of features.
3. **Averaging/Voting:** the individual predictions are combined to extract a final prediction using the mean (regression) or majority voting (classification).

These three techniques are backed by the idea of **diversity** among the trees.
A single tree is a high-variance model, so by combining many different trees, we can reduce the variance and improve the generalization performance.

$$\text{Var}\ (\bar{X}) = \frac{\text{Var}\ (X)}{T}$$

If the trees were completely independent, the variance of the mean would decrease by a factor of T (number of trees). In practice, they are not independent because the features considered are the same, so the reduction is less, but still significant.
The feature randomness further decorrelates the trees by forcing them to learn different dependencies between features. If they were all using the same features, even with different samples, they would still tend to choose the same splits.
The resulting process is:

1. For each tree t in 1 to T (number of trees, usually 100-1000):
   a. Create a bootstrap dataset D_t by sampling n instances with replacement from the original data
   b. Train a decision tree on D_t with the following constraints:
      - At each node, consider only m_try = sqrt(p) features (classification) or m_try = p / 3 (regression)
      - Grow the tree fully (no pruning) until purity (overfitting is OK, will be mitigated by bagging)

2. For prediction:
   - Classification: let all trees vote → final class == mode (most frequent)
   - Regression: let all trees predict → final value = mean of predictions

3.6.2 Time complexity

The time complexity of Random forest is based on the time complexity of training and inference of a single decision tree, multiplied by the number of trees T (Section 3.5.2).

$$O(T \cdot n \log(n) \cdot p)$$

Where n is the number of observations and p is the number of features. Note that the feature randomness reduces the effective number of features considered. Although the time complexity may initially seem high, it is important to consider that the training of multiple trees can be easily parallelized, effectively cutting down the training time. For **inference**, the time complexity again depends on the time complexity of the single tree, multiplied by T:

$$O(T \cdot d)$$

Where d is the average depth of a tree, which for unpruned trees is $d \approx \log(n)$ in the average case and n in the worst case (completely unbalanced tree). Usually, models with fast inference are preferred, even if they require longer training times; thus, training time is not a major issue, whereas inference time is more critical. In this case, Random forest is a good compromise, as it is slower than a single tree but still efficient enough for most applications.

3.6.3 Spatial complexity

The space complexity of Random forest takes into account the space needed to store the m samples and p features during training, for a total spatial complexity of $O(m \cdot p)$. Once training is completed, the space complexity is determined by the number of trees and the size of each tree:

$$O(T \cdot n)$$

3.6.4 Internal representation

A Random forest is represented as a list of trees, each of which is a complete decision tree.

```
Forest == [Tree_1, Tree_2, ..., Tree_T]

Each tree is:
Node(feature=x1, threshold=5.5, left=Node(...), right=Node(...))
```

For **interpretability**, the internal representation is a more complex and obscure structure compared to a single decision tree, making it difficult to understand the overall decision process of the forest. Similarly, its **explainability** suffers, as the local explanation of a single prediction is much more opaque for the same reason. A single tree can still be visualized and interpreted, for example by choosing the one that best explains a specific prediction, but the overall model remains of little interpretability.

3.6.5 Data assumptions

As described in Section 3.5.5, Decision Trees make very few assumptions about the data, and Random Forest inherits this property. In particular, Random Forest only assumes that the data is representative of the underlying distribution and that the features have some predictive power. The random sampling of the data must be representative to ensure that the trees learn meaningful patterns. Stratification is consequently needed to enable the trees to predict effectively.

3.6.6 Predictive performance and limitations

The ensemble nature of Random Forest allows it to achieve high predictive performance, tackling the overfitting and instability problems of a single tree. At the same time, it inherits the ability to capture complex patterns and interactions among features, as well as naturally handling both numerical and categorical features from the single tree models.
However, it has its own limitations. It is still sensitive to feature dominance, especially for categorical features with many categories. Moreover, it is not a good choice for purely linear data, where a simple linear model would achieve better performance.
The use of many trees also makes it more computationally and memory intensive. This is paired with a higher number of hyperparameters, for example the number of trees and the dimension of the random feature subset, in addition to tree depth and minimum samples per split. If not tuned properly, performance can be significantly reduced.

Finally, the output of a Random Forest is a probability (fraction of trees that vote for a class), which is not as well-calibrated as in Logistic Regression, so it may not reflect the true confidence of the prediction. To summarize, the main improvements of Random forest, better stability and lower variance, come at the cost of computational complexity, and the model still maintains some limitations with linear data, feature dominance, and class imbalance.

3.6.7 Metrics for prediction quality

To evaluate the predictive performance of Random forest, we can use both general metrics for classification and regression, as well as specific metrics that take advantage of the tree structure and ensemble nature. For the common metrics, see Section 3.1.1 (Classification) and Section 3.1.2 (Regression).

3.6.7.1 Class probability (Voting)

Random forest naturally produces a class probability for classification, based on the fraction of trees that vote for each class:

$$P(y = k) = \frac{\text{number of trees that vote for class } k}{T}$$

This probability can be used to evaluate the confidence of the prediction, but as mentioned before, it is not well calibrated, so it should be used with caution. For calibrated probabilities, post-hoc methods like Platt scaling or isotonic regression can be applied.

3.6.7.2 Out-of-Bag (OOB) Error

OOB error is a unique metric for Random Forest that provides an estimate of the generalization error without the need for a separate validation set. During training, each tree is trained on a bootstrap sample of the data, which means that some instances are left out (out-of-bag). These OOB instances can be used to test the performance of the tree on unseen data. The OOB error is calculated as the average error of the predictions made by the trees on their respective OOB instances:

$$\text{OOB Error} = \frac{1}{n} \sum_{i=1}^{n} \mathbb{1}[\text{OOB}_i \neq y_i]$$

3.6.8 Explainability and interpretability metrics

The use of plots and interpretability-oriented metrics can help to understand the behavior of the ensemble, providing a better insight into how the predictions are made.

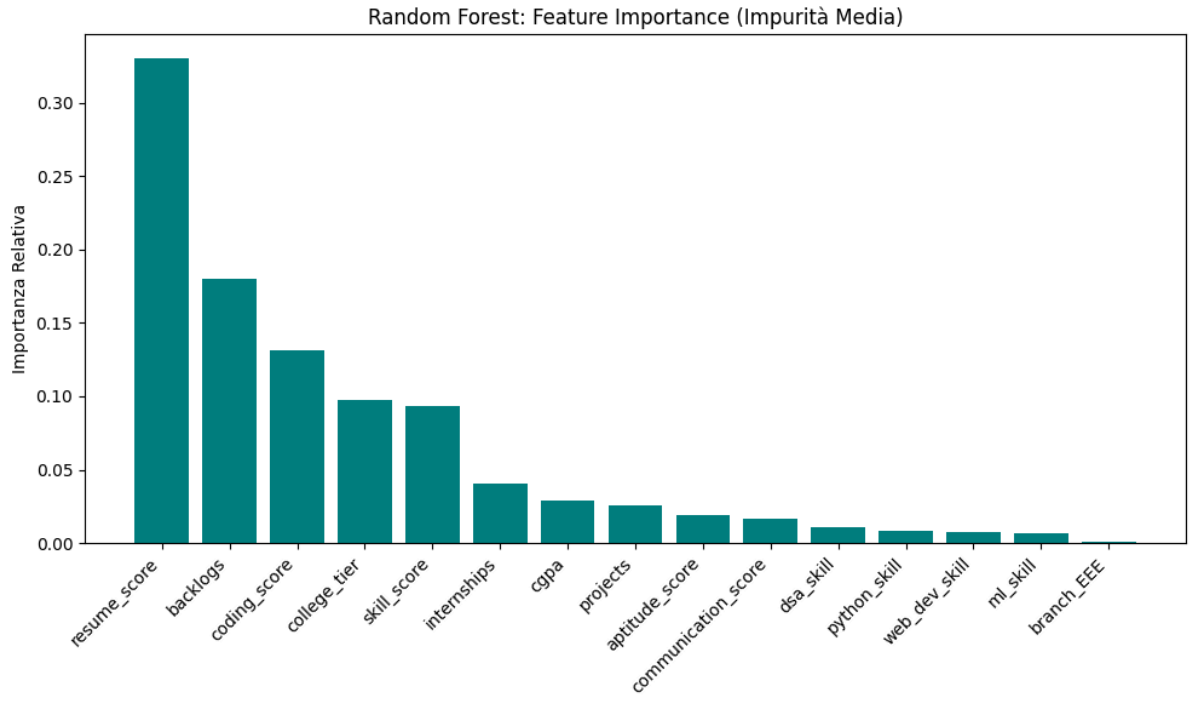
3.6.8.1 Feature importance (Average impurity)

Random Forest provides a natural way to measure feature importance based on the reduction of impurity (Information Gain) across all trees. The importance of a feature is calculated as

the average reduction in impurity that it provides when it is used for splitting, averaged over all trees in the forest:

$$\text{Importance}_j = \frac{1}{T} \sum_{t=1}^{T} \sum_{\text{nodes where feature } j \text{ splits}} \text{IG}_{t,\,\text{node}}$$

Where IG is the information gain (reduction in impurity) provided by the split on that feature at that node. This metric allows us to identify which features are most important for the predictions made by the Random Forest, and can be used for feature selection or for communicating the importance of features to non-experts, especially if visualized using bar plots.



Feature importance plot of a random forest model.

3.6.8.2 Feature Importance (Permutation-Based)

An alternative to the impurity-based method is the permutation. The idea is to randomly permute the value of a feature in the test data, measuring the degradation in performance. If the performance degrades significantly, it means that the feature is important for the model.

3.6.8.3 Partial Dependence Plot (PDP)

Shows the marginal effect of a feature on the predicted outcome, averaging out the effects of all other features. It helps to understand how the model's predictions change as a specific feature varies, while keeping other features constant. It shows how the average prediction varies as a feature varies:

$$\text{PDP}_j(x_j) == \frac{1}{n}\sum_{i==1}^{n}\hat{f}(x_j, x_{-j}^{(i)})$$

Where $x_{-j}^{(i)}$ are the values of the other features from instance i, keeping x_j fixed. Notice that if features are correlated, the effect could be misleading.

3.6.8.4 Individual Conditional Expectation (ICE) Plot

ICE plot is a variant of PDP that shows the effect of a feature on the predicted outcome for individual instances, rather than averaging over all instances. It allows us to see how the prediction changes for each instance as the feature varies, which can reveal heterogeneity in the effect of the feature across different instances.

3.6.9 Interpretability and explainability limitations

Due to its ensemble nature, Random Forest presents limitations in both explainability and interpretability, especially when compared to a single decision tree. The main issue for explainability is related to tracing the path that leads to a specific prediction. In a single tree, it is straightforward to follow the path from the root to the leaf node that makes the prediction, understanding which features and thresholds were used at each split. In a Random Forest, there are multiple trees, and each tree may use different features and thresholds for splitting, making it difficult to trace a single path for a specific prediction.
The problem is not only local, but also global, as the overall decision process is based on averaging the predictions, disrupting the interpretability of the model.
In summary, Random Forest is another example of how the trade-off between performance and interpretability is not always clear-cut, and it is important to consider the specific context and requirements of the problem when choosing a model. It does not provide a clear explanation of how the features interact to produce a specific prediction and it can be misleading in case of correlated features.

3.7 XGBoost: Extreme Gradient Boosting

3.7.1 Mathematical model

Extreme Gradient Boosting (XGBoost) is an ensemble method that uses boosting to combine the predictions of multiple decision trees. The main idea behind XGBoost is that it builds trees sequentially, training each new tree to correct the errors made by previous trees.
The resulting prediction is formulated as:

$$\hat{y}_i^{(t)} = \hat{y}_i^{(t-1)} + f_t(x_i)$$

Where $\hat{y}_i^{(t)}$ is the prediction for instance i after t trees, and $f_t(x_i)$ is the new tree that predicts the residuals of the previous iteration.
The global objective function is defined as:

$$L(\phi) = \sum_{i=1}^{n} l(\hat{y}_i, y_i) + \sum_{k=1}^{K} \Omega(f_k)$$

Where the first term is the loss function that measures the error between predicted and true values, and the second term is a regularization term that penalizes the complexity of the ensemble of K trees. Regularization is needed to prevent overfitting and is defined as:

$$\Omega(f) = \gamma T + \frac{1}{2}\lambda \sum_{j=1}^{T} w_j^2$$

penalizing both the number of leaves (T) and the leaf weight magnitude (w) using γ and λ respectively.

Considering step t, the model is trained to minimize the following objective function:

$$L^{(t)} = \sum_{i=1}^{n} l(y_i, \hat{y}_i^{(t-1)} + f_t(x_i)) + \Omega(f_t)$$

This formulation is approximated using a **second-order Taylor expansion**, resulting in a more efficient optimization process that considers both the first and second derivatives of the loss function. This also allows XGBoost to use a wider range of loss functions, with the sole constraint of being twice differentiable.

$$L^{(t)} \approx \sum_{i=1}^{n} \left[g_i f_t(x_i) + \frac{1}{2} h_i f_t^2(x_i) \right] + \gamma T + \frac{1}{2}\lambda \sum_{j=1}^{T} w_j^2$$

As said before, the gradients are defined as:

- $g_i = \frac{\partial l}{\partial \hat{y}^{(t-1)}} l(y_i, \hat{y}^{(t-1)})$ as the first-order gradient
- $h_i = \frac{\partial^2}{\partial^2 \hat{y}^{(t-1)}} l(y_i, \hat{y}^{(t-1)})$ as the second-order gradient

For the creation of the trees, XGBoost uses a greedy algorithm that iteratively splits the data based on the feature that provides the best improvement in the objective function. The quality of a split is measured by the following score:

$$\text{Score } (q) = -\frac{1}{2} \sum_{j=1}^{T} \frac{(\sum_{i \in I_j} g_i)^2}{\sum_{i \in I_j} h_i + \lambda} + \gamma T$$

Where I_j is the set of instances in leaf j. This score measures how good a tree structure is, with more negative values indicating better trees.

3.7.2 Time complexity

The time complexity for the greedy algorithm directly depends on the number of trees (K), the depth of the trees (d), and the number of observations (n). The exact greedy algorithm has a time complexity of:

$$O(K \cdot d \cdot n \cdot f \log(n))$$

Using a block structure, which moves sorting outside the tree construction without re-sorting at each iteration, the time complexity is reduced to: $O(n \cdot f \log(n)) + O(K \cdot d \cdot n \cdot f)$ Where the first term is the cost of the initial sorting and the second term is the cost of building K trees with depth d.
In reality, $n \cdot f$ is often reduced to only data entries with non-missing values thanks to the sparsity-aware algorithm, resulting in a significant speedup for sparse datasets that can amount to 50 times [16].

For **inference**, the algorithm needs to traverse K trees sequentially, following the path from root to leaf for each tree. The time complexity is therefore

$$O(K \cdot d)$$

.

3.7.3 Spatial complexity

The total spatial complexity of XGBoost is the sum of the space needed to store the model (the ensemble of trees) and the space needed to store the data during training. The spatial complexity for storing K trees with depth d is:

$$O(K \cdot 2^d + m \cdot n)$$

If a block structure is used, the spatial complexity must consider the additional space required to store column indices. This results in using double the space for the data, which does not affect big-O notation but can be significant in practice.

3.7.4 Internal representation

Being a tree-based model, XGBoost internally represents the model similarly to a random forest, as a collection of decision trees.

> Ensemble == [Tree_1, Tree_2, ..., Tree_K]
>
> Prediction == Σ_k f_k(x)

For every tree, XGBoost stores the structure of the tree (which feature is used for splitting, the threshold, and the default direction for missing values), the weights (the prediction value for each leaf), and the gradients (the accumulated gradients for each leaf during training).
For **interpretability**, even if the number of trees is typically smaller than in Random Forest (100–500 vs. 500–2000), the internal representation is still complex and not easily interpretable. The sequential nature of the trees also makes it difficult to trace which tree is responsible for a specific prediction, posing challenges for local explainability.

3.7.5 Data assumptions

The main assumption of XGBoost is that the data can be split into homogeneous classes. If this is not the case, the model may not be able to learn meaningful patterns for the minority class, resulting in poor performance. To address this issue, XGBoost provides a parameter called *scale_pos_weight* that allows developers to assign a higher weight to the minority class during training.
XGBoost also handles missing values and sparse data natively, making it robust to real-world datasets that often contain missing or incomplete information.

3.7.6 Predictive performance and limitations

XGBoost is known for its excellent predictive performance, often outperforming other machine learning algorithms in various tasks. Other advantages include robustness to missing values, the ability to handle non-linear relationships and interactions between features, and the incorporation of regularization techniques that help prevent overfitting. Its mathematical foundation provides a way to use custom loss functions in case of special requirements.
However, it is not without limitations. The model can be sensitive to hyperparameter tuning, and it may struggle with extrapolation beyond the range of the training data. Additionally, while XGBoost can handle non-linear relationships and interactions between features, it may not perform as well on datasets with a high noise-to-signal ratio or when the underlying relationship is primarily linear. Finally, its sequential nature makes it more computationally expensive, especially during training, as the boosting process itself cannot be parallelized across steps.
Nonetheless, XGBoost remains a very powerful algorithm, especially if paired with careful hyperparameter tuning, for example using Optuna (Section 4.2.5).

3.7.7 Metrics for prediction quality

To evaluate the predictive performance of XGBoost, we can use both general metrics for classification and regression, as well as specific metrics. For the common metrics, see Section 3.1.1 (Classification) and Section 3.1.2 (Regression).

3.7.7.1 Sigmoid transformation for probabilities

To obtain a calibrated probability from the margin score (the raw output of the model), we can apply the sigmoid function:

$$P(y = 1 \mid x) = \frac{1}{1 + \exp(-\text{ margin})}$$

Even if the margin score is not a true probability, this transformation allows us to interpret it as such, which can be useful for decision-making processes that require probabilities.

3.7.8 Explainability and interpretability metrics

XGBoost's intrinsic interpretability is limited, but there are some metrics that can be used to understand the importance of features (interpretability) and the contribution of each tree to predictions (explainability).

3.7.8.1 Feature Importance (Gain-Based)

Measures the average loss reduction due to each feature:

$$\text{Importance}_j = \frac{1}{K} \sum_{k=1}^{K} \text{Gain}_j^{(k)}$$

Where

$$\text{Gain}_j^{(k)}$$

is the total loss reduction when feature j splits tree k, averaged across all K trees. A feature with a higher gain is considered more important, but it is important to note that gain measurements could be biased towards features with high cardinality.

3.7.8.2 Feature Importance (Weight-Based)

Counts how many times a feature is used to split the data across all trees:

$$\text{Weight}_j = \frac{1}{K} \sum_{k=1}^{K} \#\text{ splits on feature } \ j$$

3.7.8.3 Partial Dependence Plot (PDP)

Shows how the predicted outcome changes as a single feature varies, while averaging out the effects of other features:

$$\text{PDP}_j(x_j) = \frac{1}{n} \sum_{i==1}^{n} \hat{f}(x_j, x_{-j}^{(i)})$$

The presence of particular patterns such as curves or monotonic lines shows the existence of interactions between the chosen feature and the outcome. If a flat line is observed, it suggests that the feature has little impact on the prediction.

3.7.8.4 Tree Visualization

It is possible to visualize individual trees in the ensemble, but the interpretability is limited. The scale for XGBoost is typically 100–500 trees, and visualizing all of them is impractical. Even visualizing a single tree can be complex due to the depth and number of splits, making it difficult to extract meaningful insights from the structure.

3.7.9 Interpretability and explainability limitations

Like other ensemble methods, XGBoost trades off interpretability for predictive performance. The main limitations are due to the sequential nature of the trees. A single tree cannot fully explain the prediction, and it is difficult to trace the reasoning process step by step. Even if the basic concept is easy to grasp, it is difficult to understand its application.

Additionally, the feature importance metrics can give different rankings depending on the method used (gain, weight, cover), and there is no single universally "correct" way to interpret them, as they are context-dependent. Feature importance also suffers from instability for a variety of reasons. First, correlated features can be selected alternatively, and changes in the data can affect the corresponding importance scores.

Finally, while SHAP values can provide insights into feature contributions, they are computationally expensive to calculate, especially for large datasets with many features.

To summarize, while XGBoost provides some tools for explainability, it is not designed to be an interpretable model, and its complexity can make it challenging to understand the underlying decision-making process.

3.8 Symbolic regression

3.8.1 Mathematical model

Symbolic Regression (SR) is a machine learning technique that aims to discover mathematical expressions that best fit a given dataset. Unlike traditional regression models which operate within a predefined functional form (e.g., linear or polynomial), Symbolic Regression searches over a space of mathematical expressions of varying complexity to find one that accurately describes the relationship between features and target variable.

The core idea is to represent potential solutions as expression trees, where each node represents an operation (arithmetic, trigonometric, exponential, etc.) or a feature. The goal is to evolve these expressions through a process inspired by genetic programming to minimize a loss function.

The general form of a discovered expression can be written as:

$$\hat{y} = f\big(x_1, x_2, ..., x_p\big)$$

where f is an expression discovered by the algorithm, composed of elementary operations. For example:

$$\hat{y} = x_1^2 + \sin(x_2) - 3\frac{x_3}{x_4 + 1}$$

PySR, the implementation used in this project, employs an evolutionary algorithm that maintains a population of candidate expressions and iteratively improves them through genetic operations (mutation, crossover) and selection, guided by both accuracy and complexity

(measured by the number of nodes in the expression tree). The optimization problem can be formulated as:

$$\hat{f} = \text{argmin}_f(L(f) + \lambda \text{ complexity}(f))$$

where $L(f)$ is the loss function (typically mean squared error for regression) and λ controls the trade-off between accuracy and simplicity, embodying Occam's Razor principle.

3.8.2 Time complexity

The time complexity of Symbolic Regression is highly dependent on several factors: the population size P, the number of generations G, the complexity of the expressions (number of nodes N), and the dataset size n.

For PySR specifically, each generation requires evaluating all expressions in the population on the entire dataset. Evaluating a single expression on a dataset requires traversing the expression tree for each instance, which takes $O(N)$ time per instance. Therefore, evaluating a population of size P on a dataset of size n takes:

$$O(P \cdot N \cdot n)$$

per generation

Over G generations, the total time complexity becomes:

$$O(G \cdot P \cdot N \cdot n)$$

In practice, N grows as the algorithm explores more complex expressions, making the complexity potentially very high for large populations and many generations. However, PySR implements several optimizations:

1. **Batching and Parallelization:** evaluations can be parallelized across multiple cores, reducing wall-clock time
2. **Expression Simplification:** automatic simplification of expressions reduces N during evolution
3. **Early Stopping:** if no improvement is observed over several generations, the search can be terminated early
4. **Pareto Optimization:** PySR maintains a Pareto front of expressions balancing accuracy and complexity, allowing it to discard clearly inferior expressions

For inference, the time complexity is $O(N)$ where N is the number of nodes in the final expression tree. Given that PySR favors simpler expressions, N is typically small (often less than 20 nodes), making inference very fast.

3.8.3 Spatial complexity

The spatial complexity of Symbolic Regression is dominated by the storage of the population of expressions and the dataset itself.

During evolution, PySR must store:

1. The dataset: $O(n \cdot p)$ where n is the number of instances and p is the number of features
2. The population of expressions: $O\big(P \cdot N_{\text{avg}}\big)$ where P is the population size and N_{avg} is the average number of nodes per expression
3. The history of expressions evaluated (for tracking best solutions and complexity): $O\big(P \cdot G \cdot N_{\text{avg}}\big)$ in the worst case

Therefore, the overall spatial complexity during training is:

$$O\big(n \cdot p + P \cdot G \cdot N_{\text{avg}}\big)$$

In practice, PySR optimizes this by not storing all historical expressions, resulting in a much lower actual memory footprint. The spatial complexity is further reduced by the fact that Pareto optimization allows discarding dominated solutions.

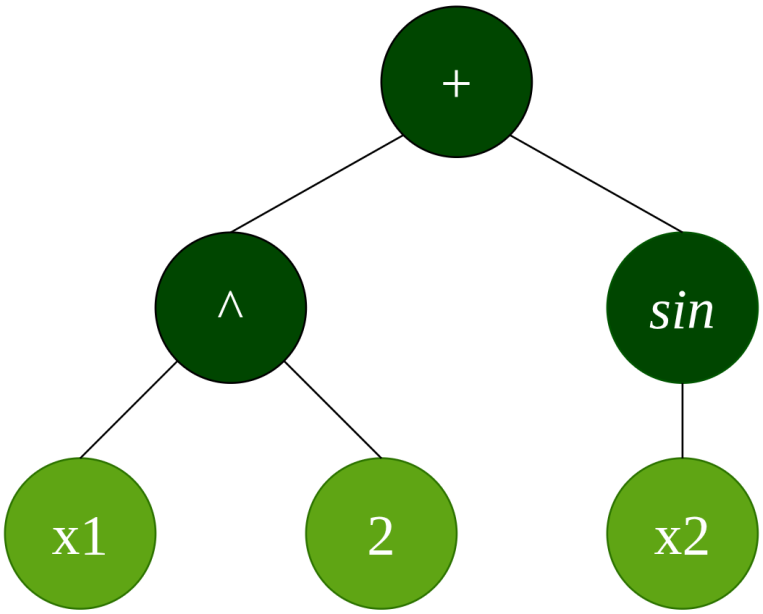
For inference, the spatial complexity is $O(N)$ for storing the final expression tree, which is typically very small (less than 100 bytes for most expressions).

3.8.4 Internal representation

A Symbolic Regression model discovered by PySR is internally represented as an expression tree. Each node in the tree represents either:

1. **Operator nodes:** arithmetic operations ($+$, -, $\times$, $\div$), trigonometric functions (sin, cos, tan), exponential/logarithmic functions (exp, log), power operations, etc.
2. **Leaf nodes:** features (x_i) or constants (real numbers)

For example, the expression $\hat{y} = x_1^2 + \sin(x_2)$ would be represented as:



Expression tree representation of a symbolic regression model.

This representation has profound implications for **interpretability**. Unlike black-box models, the discovered expression is fully transparent and interpretable. A domain expert can directly read the mathematical relationship between features and the target variable, understand how features interact, and potentially derive new insights about the underlying system.

The explicit mathematical form also allows for analytical operations such as computing derivatives, identifying discontinuities, or analyzing asymptotic behavior.

However, as expressions become more complex (more nodes in the tree), interpretability degrades. This is why PySR explicitly penalizes expression complexity during evolution, maintaining a trade-off between accuracy and interpretability (the Pareto front).

3.8.5 Data assumptions

Symbolic Regression makes very few structural assumptions about the data compared to traditional regression models. However, several practical considerations exist.

While SR can discover non-smooth relationships, it performs best when the underlying relationship is relatively smooth. Highly discontinuous or noise-dominated relationships are difficult to capture.

Additionally, features should ideally be scaled to similar ranges. Highly disparate scales can lead to numerical instability during expression evaluation and can bias the algorithm towards features with larger magnitudes.

1. **Noise Sensitivity:** SR can be sensitive to noise in the data. High noise levels can lead to overfitting, where the discovered expression fits noise rather than the underlying pattern. This is mitigated by the complexity penalty in PySR.

In addition, a reasonable amount of data is needed for reliable discovery. With very small datasets, the discovered expressions may not generalize well.

1. **Feature Relevance:** irrelevant features in the input can mislead the algorithm. Feature selection or using domain knowledge about relevant features improves results.

Unlike linear models, SR does not suffer from strong assumptions such as linearity of relationships, homoscedasticity, normal distribution of residuals, or independence of features. This flexibility makes SR applicable to a wide range of problems where traditional assumptions would be violated.

3.8.6 Predictive performance and limitations

Symbolic Regression offers several advantages in terms of predictive performance.

It is an extremely flexible method that can discover complex non-linear relationships of arbitrary functional form. Feature transformations (polynomials, logarithms, trigonometric functions) are discovered automatically, without manual specification. Additionally, the complexity penalty encourages simpler models that often generalize better than complex black-box models.

On the other hand, Symbolic Regression suffers from several important limitations.

First of all, the search process is computationally expensive, especially for large datasets or high-dimensional feature spaces. Training can take minutes to hours, even with parallelization. Second, the space of possible expressions is extremely large, and there is no guarantee of finding the global optimum.
Third, discovered expressions may behave unexpectedly outside the range of training data, as they are not constrained by physical laws.
Fourth, PySR has many hyperparameters (population size, generations, complexity penalty, mutation rates) that significantly affect results and require careful tuning. Fifth, discovering meaningful expressions requires high-quality, relatively noise-free data. Poor data quality leads to spurious expressions. Finally, performance degrades significantly with very high-dimensional feature spaces, as the search space becomes prohibitively large.
Consequently, it is best suited for problems where the underlying relationship is suspected to have a relatively simple mathematical form, interpretability is important, computational resources are available, and data quality is good with a moderate dataset size. Its power has been directly demonstrated in scientific discovery, in particular in physics [17].

3.8.7 Metrics for prediction quality

For evaluating the predictive performance of Symbolic Regression, standard regression metrics apply. See Section 3.1.2 for general metrics such as RMSE, MAE, and R^2.
Additionally, SR-specific metrics are valuable:

3.8.7.1 Complexity

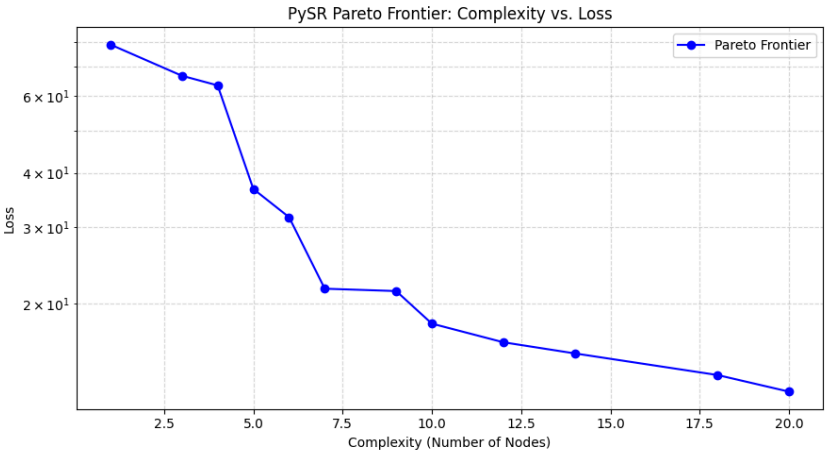
The complexity of an expression is typically measured as the number of nodes in the expression tree N. PySR uses this metric as part of its fitness function to prefer simpler expressions:

$$\text{Complexity} = N$$

A smaller complexity indicates a simpler, more interpretable expression. The trade-off between complexity and accuracy is explicitly managed through the complexity penalty parameter in PySR.

3.8.7.2 Pareto Front Analysis

PySR maintains a Pareto front of non-dominated expressions, expressions where no other expression is simultaneously simpler and more accurate. This provides the user with a set of candidates to choose from based on their specific accuracy-interpretability trade-off preference.



Pareto front of discovered expressions showing the trade-off between loss and complexity.

3.8.7.3 Expression Stability

Measures how similar the discovered expressions are across multiple runs with different random seeds. More stable expressions (discovered consistently) are generally more trustworthy than those that appear by chance.

3.8.8 Explainability and interpretability metrics

The primary advantage of Symbolic Regression is that the model itself is an explanation. However, several metrics help quantify and visualize feature importance and expression behavior.

3.8.8.1 Feature Importance from Expression Analysis

Since the discovered expression is explicit, feature importance can be computed directly by analyzing the structure:

1. **Frequency Analysis:** count how often each feature appears in the expression tree. Features appearing multiple times or in critical positions (e.g., as the argument to exponential functions) are more important.
2. **Sensitivity Analysis:** for a given instance, compute the partial derivative of the expression with respect to each feature:

$$\text{Importance}_j^{(i)} = \mid \frac{\partial f}{\partial x_j}(x^{(i)}) \mid$$

This measures how much the prediction would change with a small change in each feature, providing an instance-level feature importance.

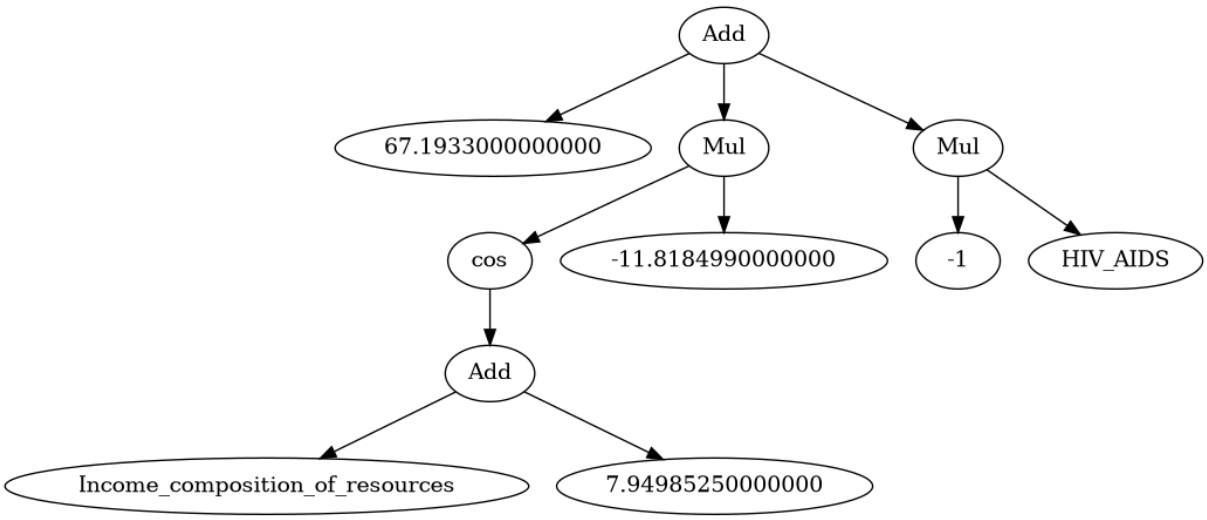
1. **Feature Interaction Identification:** inspect the expression structure to identify which features interact multiplicatively, additively, or through other operations, providing insight into feature relationships that the model has discovered.

3.8.8.2 Sensitivity and Derivative Analysis

Since the discovered expression is differentiable (except at discontinuities), one can perform sensitivity analysis by computing derivatives with respect to each feature. This allows for understanding how small changes in input features affect the output prediction and model behavior.

3.8.8.3 Expression Complexity Visualization

As shown in Section 3.8.4, the discovered expression can be visualized as an expression tree. This visualization provides insight into the structure of the model and how it was constructed.



Expression tree representation of a symbolic regression model using the life expectancy dataset.

3.8.9 Explainability limitations

While Symbolic Regression provides superior interpretability compared to black-box models, it has specific limitations. Even with the complexity penalty, discovered expressions can become quite complex (50+ nodes), at which point they become difficult for humans to understand and may represent overfitting rather than true underlying relationships.

While one can understand the entire equation globally, understanding **why** specific feature interactions emerge can be difficult without domain expertise. Multiple mathematically equivalent expressions can exist (e.g., $x^2 + 2x + 1$ vs $(x + 1)^2$), and the algorithm may discover one that is less intuitive than another.

Expressions involving operations like absolute value or maximum can contain **discontinuities**, making them difficult to analyze and potentially problematic for certain applications.

Despite these limitations, Symbolic Regression remains one of the most interpretable machine learning approaches available, making the discovered **explicit mathematical relationships** valuable for scientific discovery and understanding of complex systems.

3.9 Algorithmic comparison

3.9.1 Performance comparison

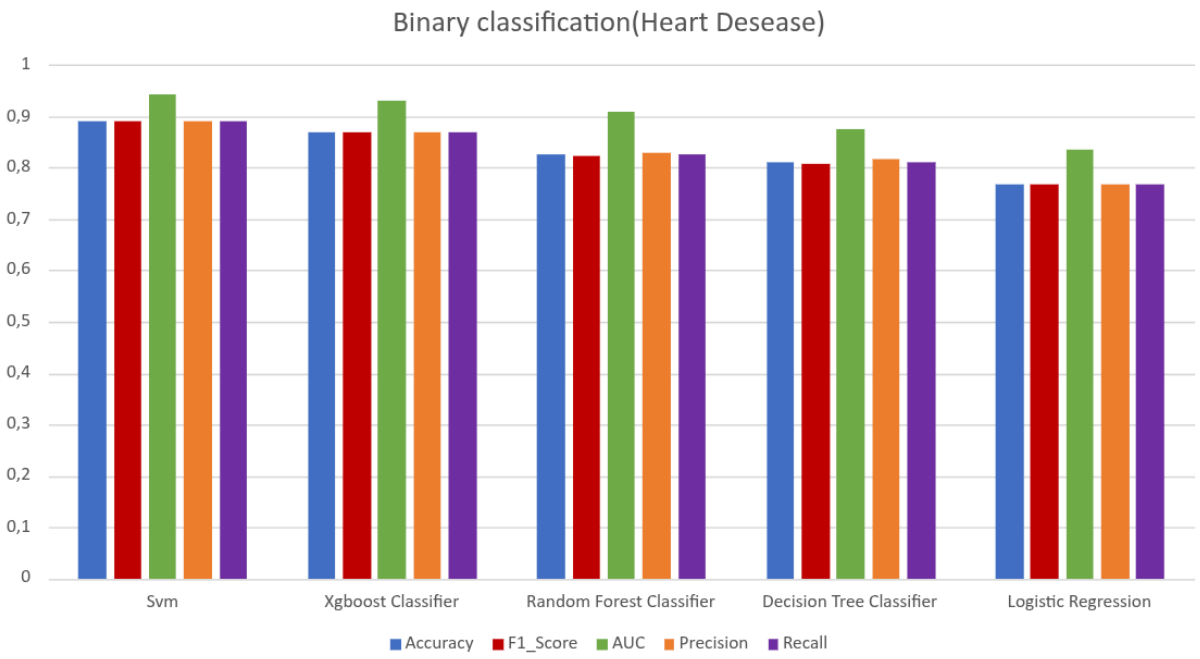
To compare the performance of the different algorithms, the following metrics were used:

- For classification algorithms: accuracy, F1-Score, AUC, precision, and recall.
- For regression: R^2, Adjusted R^2, MAE, MSE, and RMSE.

As a dataset, in the initial phase, a synthetic dataset retrieved from Kaggle was used (Student Placement & Salary Dataset [1]) for both regression and classification tasks. Then, the same metrics were used to compare the algorithms on the real datasets used in the subsequent evaluation.

For all algorithms, the best hyperparameters are automatically chosen using Optuna and its Bayesian optimization approach, which enables efficient exploration of the hyperparameter space and finds the optimal configuration for each algorithm for a specific dataset.

Binary classification For the binary classification task, the heart disease dataset [3] was used, which contains 12 features and a binary target variable indicating the presence or absence of heart disease. The performance comparison of the algorithms is shown in the following figures.



Binary Classification performance comparison of the algorithms on the heart disease dataset.

As shown in the figure, the best performing algorithm is SVM, achieving an F1-score of 0.8909, followed by XGBoost with an F1-score of 0.8695, and Random Forest with an F1-score of 0.8245. The Decision Tree performs the worst, with an F1-score of 0.7667.

A similar trend is observed for the other metrics, with SVM consistently outperforming the other algorithms across all metrics, followed by XGBoost and Random Forest, while Decision Tree performs the worst.

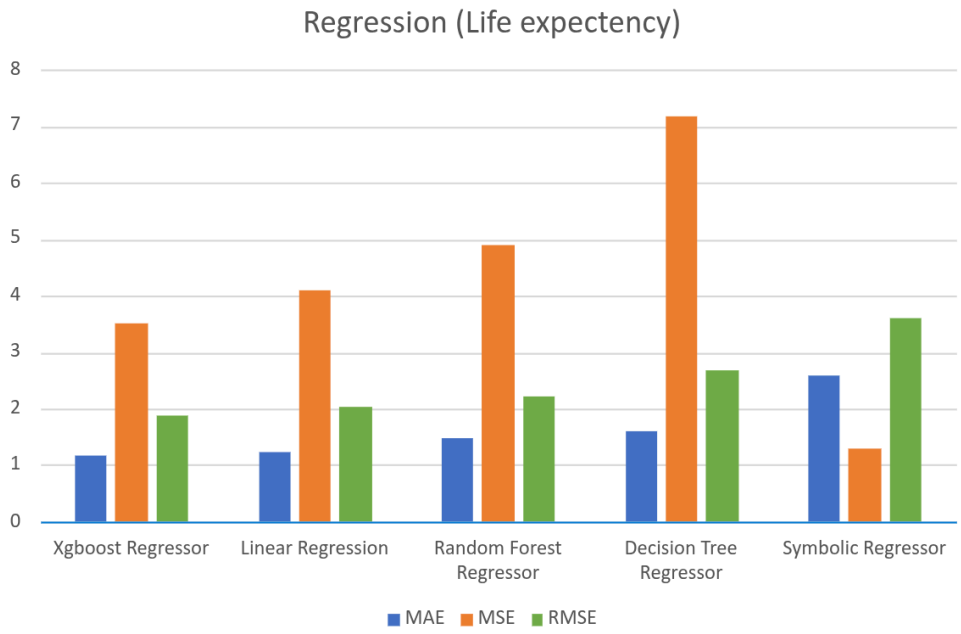
What emerges is that the ensemble methods (Random Forest and XGBoost) perform better than the single decision tree. This is expected, as ensemble methods combine multiple trees to reduce overfitting and improve generalization, while a single decision tree can easily overfit the training data.

Logistic regression, on the other hand, performs worse than the others; this is also expected, as it is a linear model and the relationships between the features and the target variable are likely non-linear.

Finally, it is interesting to notice that the SVM performs better than the other algorithms, which can be explained by the fact that it finds a non-linear decision boundary using kernel functions, allowing it to capture complex relationships between features and the target variable.

Regression For the regression task, the life expectancy dataset [2] was used, which contains 22 features and a continuous target variable representing life expectancy. The performance comparison of the algorithms is shown in the following figures.

Regression performance comparison of the algorithms on the life expectancy dataset.



Regression performance comparison of the algorithms on the life expectancy dataset with focus on error metrics.

Similar results are observed for the regression task, with the XGBoost algorithm outperforming the other algorithms across all metrics, achieving an adjusted R^2 of 0.9089, followed by linear regression with 0.8939, Random Forest with 0.8730, and Decision Tree with 0.8143. Symbolic regression performs the worst with an adjusted R^2 of 0.6624.

Again, the ensemble methods perform significantly better than the single decision tree, which is known to underperform in regression tasks.

Interestingly, a simple model such as linear regression achieves very good results with predictions close to those provided by more complex models. This is due to the nature of the problem and the dataset, but it is important to underline how a simple model can still achieve good performance in certain real-world scenarios.

Symbolic regression performs the worst, which is expected as it is a model that can easily overfit training data if not properly regularized, especially when the dataset is small or noisy.

3.9.2 Interpretability and explainability vs. performance comparison

An interesting yet intuitive pattern emerges when comparing the performance of the algorithms with their interpretability and explainability. In general, more complex models tend to perform better than simpler models, but they are also less interpretable and harder to explain.

Ensemble methods show a substantial boost in predictive performance compared to single tree models, but they also result in a more obscure decision-making process, making it difficult to understand how the model works (interpretability) or why it makes specific predictions (explainability).

For this exact reason, it is important to consider the context and specific requirements of the

problem when choosing an algorithm. In some cases, a simpler model with lower performance may be preferred if interpretability and explainability are crucial, while in other cases, a more complex model with higher performance may be acceptable if accuracy is the primary concern. Post-hoc explainability techniques can support complex models by explaining why a specific outcome occurred, compensating for their lack of transparency. However, this comes at an additional computational cost that can be significant. For example, when computing SHAP values for SVM, the computational price is higher than for tree-based models because it cannot leverage the internal structure of trees. The values are calculated by perturbing the input features and observing the change in the model's output, which can be computationally expensive, especially for large datasets.

In conclusion, the choice of algorithm should be guided by a careful consideration of the trade-off between performance, interpretability, and explainability, as well as the specific requirements of the problem at hand, leveraging post-hoc explainability techniques when necessary to explain the outcomes of complex models.

Chapter 4

Design and code implementation

In this chapter we will describe the design and the implementation of the system, starting from the requirements and the technologies used, to the design and coding of the system.

4.1 Requirements

Before implementing the system, it is crucial to define the goals and requirements of the project. For a more precise description, these requirements are categorized into functional (F), qualitative (Q), and constraint (V) requirements, and each of them is classified as necessary (N), desirable (D), or optional (O).

Requirement	Description
RFN-1	The system must allow the user to choose the dataset to analyse
RFN-2	The system must allow the user to choose the algorithm for the analysis
RFN-3	The system must allow the user to choose the level of detail for the analysis (example SHAP/No-SHAP)
RFN-4	The system must allow the user to choose whether to execute the LLM analysis or not
RFN-5	The system must allow the user to access the result of the analysis in a clear and organized way
RFN-6	The system must allow the user to know the reason in case of an error during the execution of the analysis
RFN-7	The system must allow the user to run multiple analyses to compare the performances of different algorithms and techniques
RFO-1	The system should be accessible via Graphical User Interface (GUI), not only Command Line Interface (CLI)

Requirement	Description
RQN-1	The system should be open to different datasets, algorithms
RQN-2	The system should be open to the use of different explainability and analysis techniques
RQN-3	The system should be open to the use of different LLMs for the analysis
RQD-1	The system should process the data and execute the analysis in a reasonable time frame, including the LLM analysis, the pipeline should not take more than 15 minutes to execute

Table of requirements for the analysis pipeline.

4.2 Technologies and tools

Considering the requirements defined in the previous section and the context of the project, the following technologies and tools were chosen for the implementation of the system:

4.2.1 Python

Python has been chosen as the main programming language for the implementation of the system due to its versatility, ease of use, and the wide range of libraries and frameworks available for data analysis, machine learning, and natural language processing.

4.2.2 Markdown

Markdown has been chosen for the collection of the LLM responses to generate the final report. It was chosen for its simplicity and readability, as well as its compatibility with various tools and platforms for documentation and reporting.

4.2.3 Pandas, Matplotlib, Seaborn, Numpy, SHAP

These Python libraries have been chosen for the vast range of functionalities. Pandas for data manipulation and analysis, Matplotlib and Seaborn for data visualization, Numpy for numerical computations, and SHAP for explainability analysis (Section 2.2.3).

4.2.4 Scikit-learn

Scikit-learn provides implementations of most of the algorithms under consideration without the need to implement them from scratch, allowing the developer to focus on the analysis and explainability aspects of the project.
Moreover, it provides a variety of metrics for evaluating the performance of the models, which is crucial for the analysis.

The models that were not offered by scikit-learn were implemented using the original implementations provided by the authors of the algorithms, for example, for XGBoost [16].

4.2.5 Optuna

Since many algorithms benefit from hyperparameter tuning, Optuna was chosen for its efficiency and ease of use in automatically optimizing the hyperparameters of machine learning models.

4.2.6 Streamlit

Streamlit was chosen for the implementation of the GUI of the system. It enables quick and easy creation of interactive web applications for data analysis and machine learning, which is essential for providing a user-friendly interface for the system.

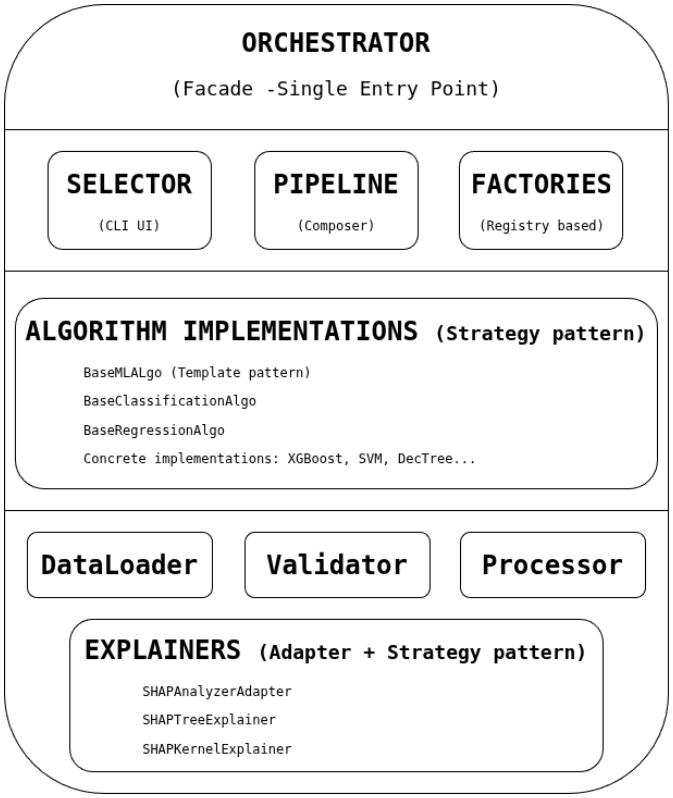
4.2.7 GitHub

GitHub was chosen for the version control of the project, for an effective codebase management.

4.2.8 Typst

Typst was chosen as primary tool for the notes and documentation, thanks to its flexibility, modern design, powerful features and effective rendering of the PDFs.

4.3 Design



The design of the entire pipeline was made with extensibility in mind, providing the developer with the capability to easily add new algorithms, datasets, analysis techniques, and LLMs.

For this reason, the design of the system is layered and modular, with each component of the pipeline being independent and interchangeable.

The main components of the system are:

Diagram of the layered architecture of the system with the most relevant components.

1. **Orchestrator**: coordinates the pipeline, invoking the different components in the correct order and passing the necessary data between them.
2. **Selector**: allows the user to choose the analysis type (single algorithm or comparative), the dataset, the algorithm, the level of detail for the analysis and whether to execute the LLM analysis or not.
3. **Algorithm**: implements the machine learning algorithm and provides the necessary functionalities for training, prediction and evaluation. All the models are created using a factory pattern, enabling the easy addition of new algorithms without modifying the existing code.
4. **Data Pipeline**: responsible for loading, preprocessing and splitting the data for the analysis. Every single responsibility is encapsulated in a specific class.
5. **LLM Request Manager**: manages the requests to the LLM, including the generation of prompts and the collection of responses.

4.3.1 Guiding principles

To achieve the best possible design for the system, object-oriented programming principles and design patterns were applied. First, the SOLID principles [18], [19] were applied. In particular:

1. **Single Responsibility Principle**: each class has a single responsibility, making the code more modular and easier to maintain. A clear example is the modular structure cited just before. A concrete example is the management of the dataset. The *DataPipeline* interface describes the structure of the data pipeline, while the *DatasetLoader* interface is responsible for loading the dataset, the *DataValidator* interface is responsible for validating the data, the *DataProcessor* interface is responsible for preprocessing the data and the *DataSplitter* interface is responsible for splitting the data into training and test sets.
   This separation allows the developer to replace or modify each component without affecting the others, making the code more **flexible** and **maintainable**.
2. **Open/Closed Principle**: each implementation of the abstract *baseMLAlgo* can expand the capabilities of the base class, adding new functionality without modifying the existing code. All the implementations inherit the basic skeleton of the base class, wrapping the specific algorithm, usually a scikit-learn estimator. The algorithm implementation only needs to implement the specific methods for fitting and metrics/plot generation, following the template defined by the base class. This allows the system to easily add new algorithms without modifying the existing code, making the system more **extensible** and **scalable**.
3. **Liskov Substitution Principle**: the implementations of *baseMLAlgo* substitute the abstract class, applying their version of the functions and modifying the analysis. The *orchestrator* uses the abstract class, allowing the orchestrator to easily switch between

different implementations of the algorithms without modifying its own structure. The same principle has been applied in the *DataPipeline*, providing a clear interface for data loading, validation, processing, and splitting, which enables easily **switching between different implementations** of these components without affecting the rest of the system.

4. **Interface Segregation Principle**: the interfaces of the different components are designed to be specific to their functionality, avoiding unnecessary dependencies between them. For example, the *LLMRequestManager* has a specific interface for managing requests to the LLM, without depending on the implementation of the algorithms or the orchestrator. The same applies to the *DataPipeline* and *Explainer* components, which are designed to be independent and modular, allowing developers to easily replace or modify each component without affecting the others. This design promotes **loose coupling** and **high cohesion** in the codebase, making it easier to maintain and extend.
5. **Dependency Inversion Principle**: high-level modules (*orchestrator*) do not depend on low-level modules (algorithms, LLM request manager); instead, both depend on abstractions. For example, the orchestrator depends on the abstract *baseMLAlgo* and *LLMRequestManager*, allowing it to switch easily between different implementations of the algorithms and the LLM request manager without modifying its core structure.

4.3.2 Applied design patterns

The design of the system also incorporates some design patterns to solve common problems and improve the structure of the code. In particular, the following design patterns were applied:

4.3.2.1 Strategy

The strategy pattern enables switching between different algorithms without modifying the code of the *orchestrator*. Each algorithm implements the same interface defined by the abstract *baseMLAlgo*, selecting the appropriate strategy at runtime.

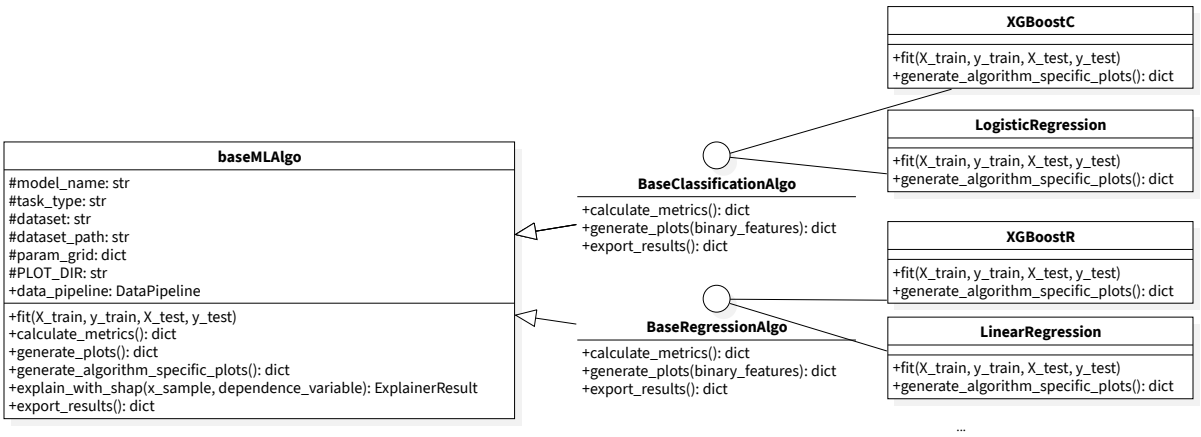


Diagram of the strategy pattern applied to the implementation of the algorithms with XGBoost, Logistic and linear regression as examples of the concrete algorithms.

4.3.2.2 Template method

BasePipeline defines the overall structure of the analysis process. The concrete *DefaultPipeline* implements the template method, providing specific implementations for each step. The same pattern was used in the *DataPipeline* and *Explainer* components, as they share the same requirement for a defined structure with the possibility of customizing specific steps.

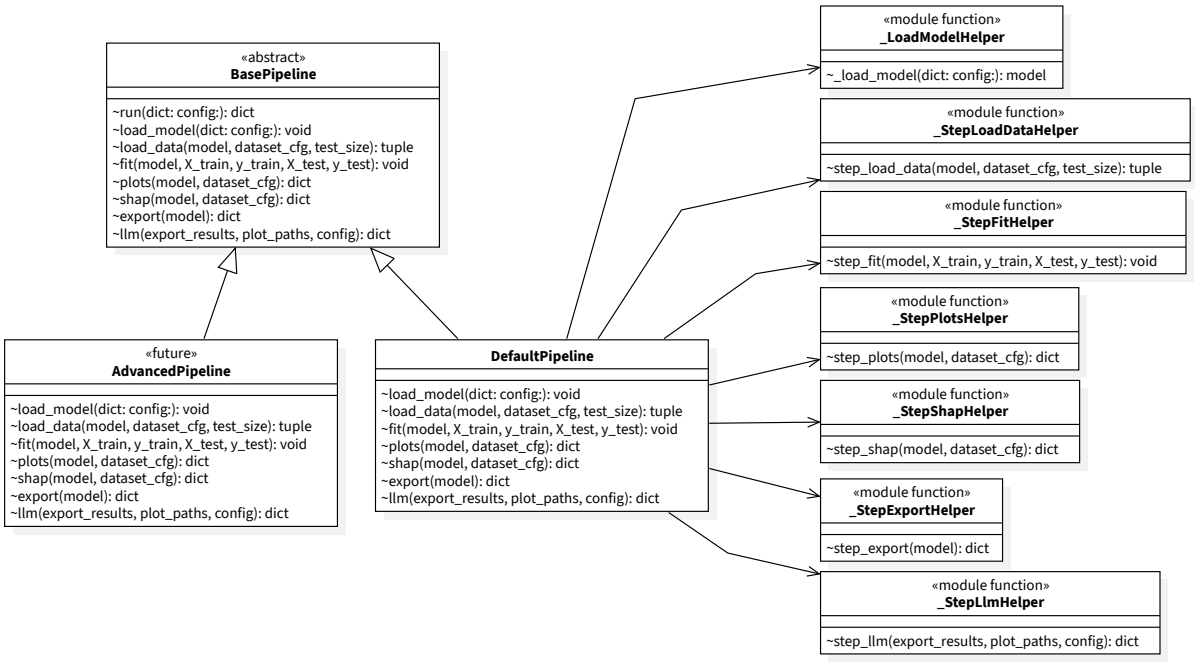


Diagram of the template method pattern applied to the implementation of the pipeline.

4.3.2.3 Factory

The factory pattern was applied in the algorithms instantiation via *ModelFactory*, allowing developers to easily create instances of the algorithms without exposing the instantiation logic to the client code. This makes it easy to add new algorithms without modifying the existing code and dynamically load them, making the system more **extensible** and **scalable**.

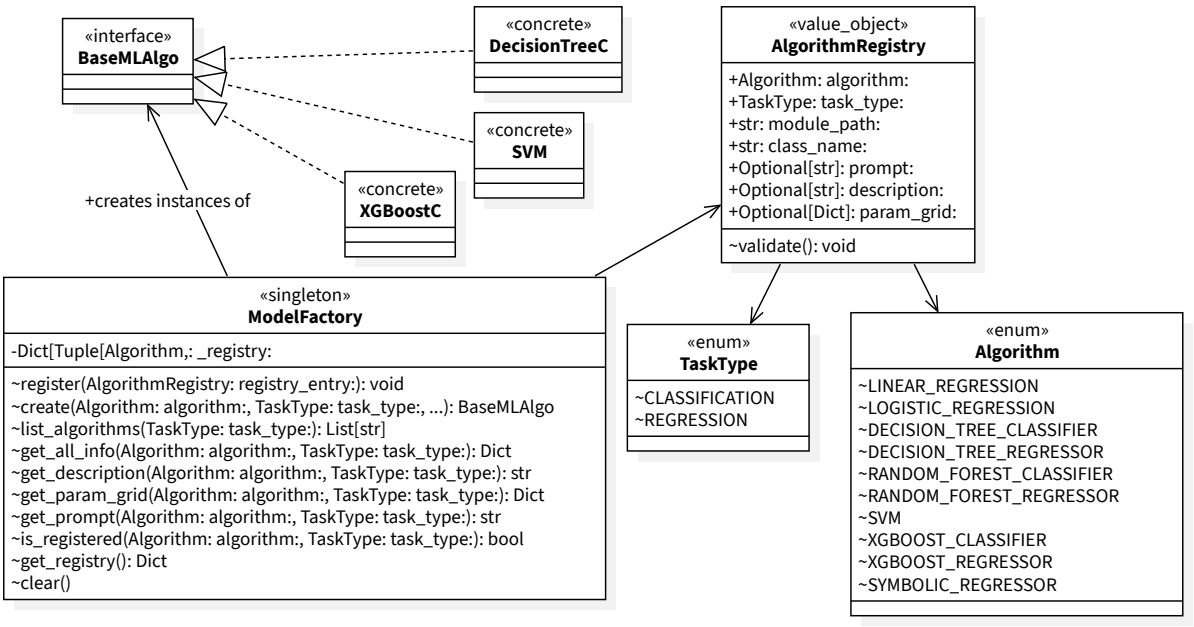


Diagram of the factory pattern applied to the implementation of the algorithms and ModelFactory.

4.3.2.4 Registry

To centrally manage the registered algorithms, a registry pattern was applied, enhanced by the extensive use of value objects to manage algorithm information. The validity of the registered algorithms is consequently guaranteed, simplifying the process of adding new algorithms to the system, as it only requires creating a new class that implements the *baseMLAlgo* interface and registering it in the *ModelFactory* without modifying the existing code. This design promotes **extensibility** and **maintainability** of the codebase.

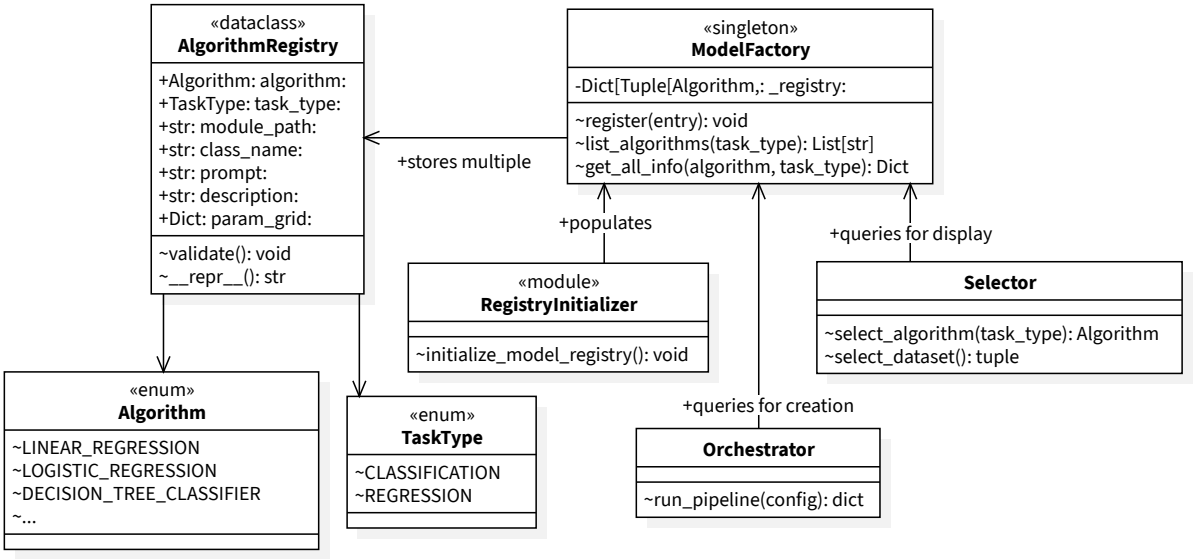


Diagram of the registry pattern applied to the implementation of the algorithms and its use in the context of the system.

4.3.2.5 Adapter

Computing SHAP values for the algorithms requires some adaptation to fit the specific requirements of the SHAP library. For this reason, an adapter pattern was applied to the *Explainer*, allowing the system to easily adapt the algorithms to the requirements of the SHAP library without modifying the existing code while maintaining a consistent interface. This design promotes **flexibility** and **reusability** of the codebase.

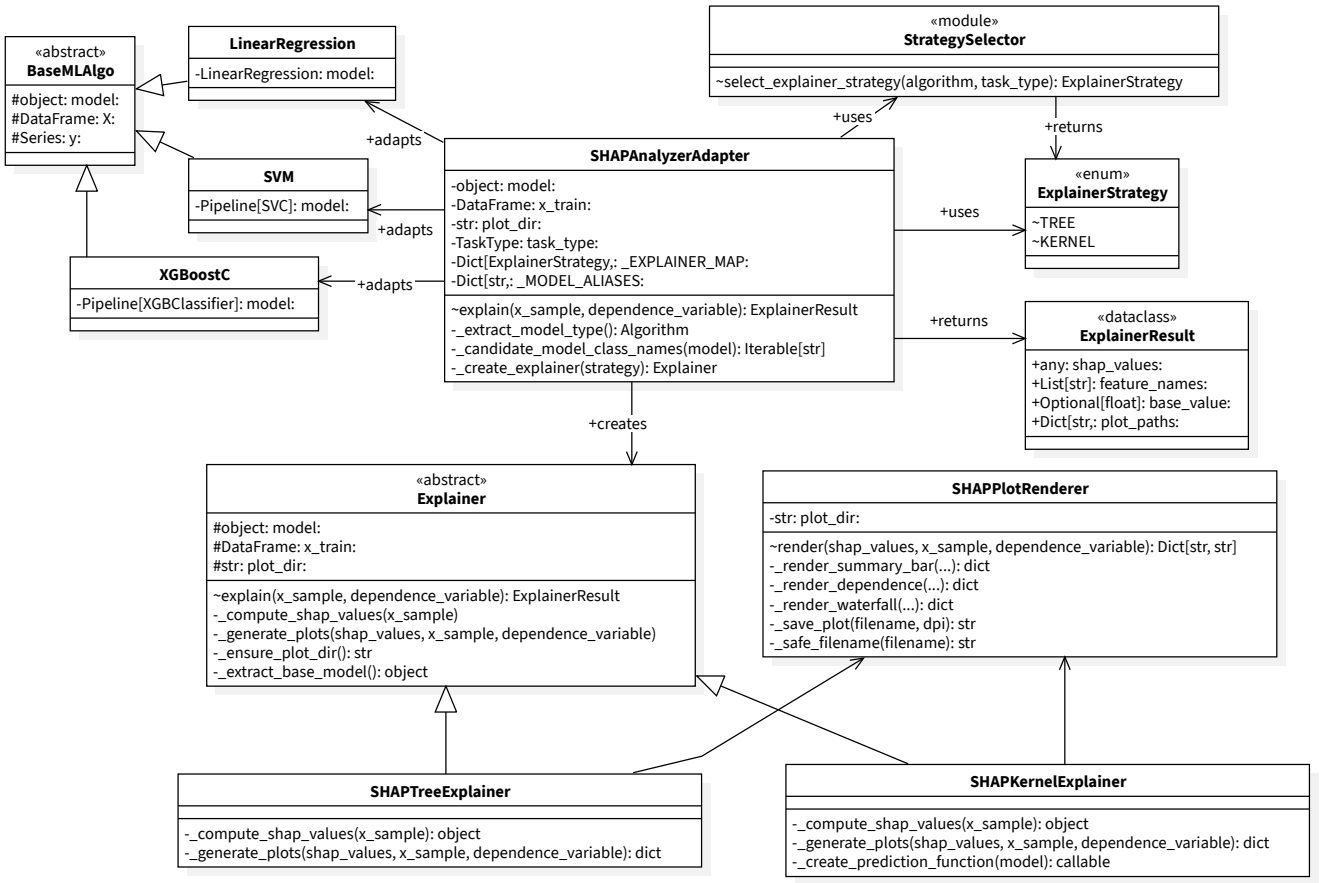


Diagram of the adapter pattern applied to the implementation of the algorithms and its use in the context of the system.

4.3.2.6 Facade

The facade pattern was applied to the *orchestrator*, providing a single entry point *run_pipeline()* for algorithm initialization, result aggregation, and management of the entire process of generating prompts, sending requests to the LLM, and collecting responses, hiding the complexity of the underlying implementation from the rest of the system.

4.4 Code implementation

Consequently, the resulting product of the design process is structured in a modular and layered way, with clear interfaces and separation of concerns among the different components.

The implementation of the system follows the design principles and patterns described in the previous sections, resulting in a codebase that is **extensible**, **maintainable**, and **scalable**.

4.4.1 Extension of the system

The process of extending the system with new algorithms, datasets, and analysis techniques follows a clear and structured process, enabling the easy addition of new components to the system without modifying the existing code. The process is as follows:

1. **New algorithm**

STEP 1

NewAlgo.py

Describe the new model following "baseMLAlgo" interface and its template method using the abstract classes as guidelines.

STEP 2

registryInitializer.py

Register the new algorithm in the "RegistryInitializer" to make it available in the system.

STEP 3

Explainer.py

Add an entry in the "Explainer" strategy using the updated registry for type safety.

2. **New dataset manipulation**

dataset_config.py

If what is being added is a new dataset, create a new entry in the "dataset_config", describing the dataset, the task, the objective and other relevant information for the analysis.

dataLoader.py

If what is being added is a new dataset source, create a new class that implements the "DataLoader" interface, providing the necessary methods for loading the new dataset.

dataValidator.py

If a validation rule is being added, create a new class that implements the "DataValidator" interface, providing the necessary methods for validating the new dataset.

dataProcessor.py

If what is being added is a new preprocessing step, create a new class that implements the "DataProcessor" interface, providing the necessary methods for preprocessing the new dataset.

3. **New SHAP analysis technique**

STEP 1

NewSHAP.py

Create a new class that implements the "SHAPExplainer" interface, providing the necessary methods for computing the SHAP values using the new technique.

STEP 2

strategy.py

Define in the "strategy" which algorithms can use the new SHAP technique, using the algorithm registry for type safety.

STEP 3

plot_renderer.py

Add any new methods for rendering the SHAP values.

4.4.2 Flow of the system

For an understanding of how the system works, the flow of the pipeline is described in the following diagram, showing the main steps of the process and how the different components interact with each other.

As described before, the *orchestrator* controls the execution of the pipeline, invoking the *selector* to get the user input. Once the configuration for the run is ready, the analysis starts. For every algorithm requested by the user, the *ModelFactory* is used to create an instance of the algorithm. The *DataPipeline* is used to load, validate, preprocess and split the data for the analysis. The algorithm is trained and evaluated, and the metrics are collected in a results container. If the user requested SHAP analysis, the *Explainer* component selects the appropriate explainer, computes the SHAP values for the algorithm, and generates the relevant plots. The results are exported in a single directory created at the start of the analysis. Finally, if the user requested the LLM analysis, the *LLMRequestManager* is used to generate the prompts, send the requests to the LLM and collect the responses, which are then aggregated in a final report in Markdown format.

If the user initially requested a comparative analysis, the metrics are collected in a table and displayed.

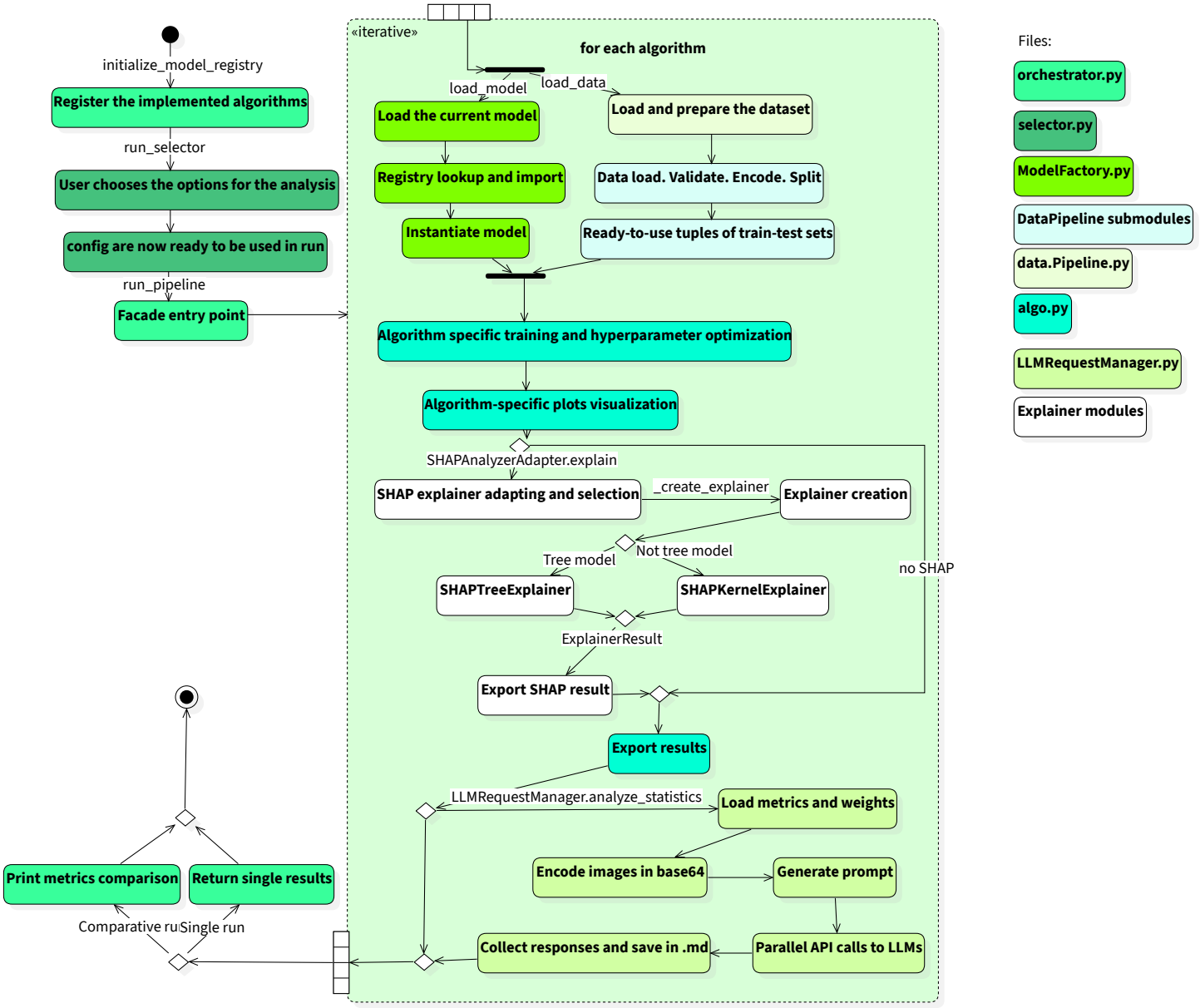


Diagram of the flow of the system, showing the main steps of the process and how the different components interact with each other.

Chapter 5

Prompt Engineering

In this chapter, we will expose how the prompting principles are applied, how the prompt system is organized and the principal results obtained from this process.

5.1 Prompt structure

The structure of the prompt is divided into four main sections:

1. **General instructions and guidelines**: this section contains the general instructions for the LLM, including the task description, the expected output format, and any specific guidelines for generating explanations.
2. **Algorithm specific instructions**: this section contains the instructions for the specific algorithm being used.
3. **User query**: this section contains the specific question or request for explanation that the LLM will respond to.
4. **Payload**: this final section contains both the metrics, the coefficients and the base64 encoded images.

```python
prompt_text = (
        f"Analysis context:\n\n"
        f"# ALGORITHM: {algo_name}\n"
        f"## Algorithm type: {algo_type}\n"
        f"## Dataset description: {dataset_description}\n"
        f"## User expectations: {user_prompt}\n\n"
        f"# NUMERICAL DATA:\n{raw_metrics}\n\n"
        f"# COEFFICIENTS: \n{raw_coefficients}\n\n"
        f"# SPECIFIC INSTRUCTIONS: {algo_prompt}\n"
        f"{general_prompt}\n"
    )
```

5.2 General instructions and guidelines

This section of the prompt is designed to describe the guidelines for the analysis. This includes not only the instructions for the task and the style of the output, but also the principles of explainability that the LLM should follow when generating the explanation, as described in Section 2.2.

```python
general_prompt = (
        "# STRICT EXPLAINABILITY RULES (Rigorous guidelines):\n"
        "1. Simplicity and Brevity: Focus only on a limited number of determining causes (maximum 3-5 features). Ignore marginal or non-causal variables.\n"
        "2. Fidelity to Human Logic: Use human language consistent with the business domain, adapting the
```

```
explanation to the audience.\n"
        "3. Causality vs Correlation: Always clarify that the features identified by the model indicate statistical
correlations but do not necessarily imply direct causality.\n"
        "4. Anomaly Management and Feature Support: If you notice extreme values (outliers) or if the
prediction seems to be based on unusual data, report that the reliability (support) could be low as it deviates
from average cases.\n"
        "5. Contrastive Explanation: If the data and image allow, try to explain differences contrastively (e.g.,
'why the instance is positive compared to the negative one').\n\n"
        "# GENERAL INSTRUCTIONS:\n"
        "1. Summarize the overall reliability of the model in a few steps, based on the data but without going
into technical details.\n"
        "2. Intuitively explain the main factors (features) driving decisions.\n"
        "3. Analyze the data and attached graphs to confirm if the model's decisions are in line with business
common sense."
        "4. If the data contradicts user expectations or if anomalies emerge, highlight these discrepancies and
suggest possible interpretations or corrective actions.\n"
        "# CONSTRAINTS:\n"
        "- Avoid excessive technical jargon, aim for clear and accessible language.\n"
        "- Do not just repeat the data, but provide an interpretation that makes them understandable and
useful for strategic decisions."
        "- No explicit mathematical formulas."
        )
```

The direct inclusion of the explainability enforces the model to follow patterns proved to be effective for human understanding, avoiding the use of technical jargon and the inclusion of irrelevant features, which are in contrast with the context of the end user, who is a business user with limited technical knowledge. It is important to notice that there is a direct numeric indication of the number of features considered. This prevents the model from taking too much freedom in the choice of the level of detail of the explanation.
Another important aspect is the inclusion of the point 4. *"If the data contradicts user expectations or if anomalies emerge, highlight these discrepancies and suggest possible interpretations or corrective actions."*. This forces the model to provide objective explanations, controlling the so called **"temperature"** of the model. This is a crucial point, as it prevents the model from providing explanations that are in line with user expectations but not supported by the data.

5.3 Dataset integration

The dataset is integrated into the prompt not only through the raw coefficients and metrics but also through the description of the dataset. This provides the necessary context to link the metrics, the coefficients, the user expectations, and the images to a pragmatic business context. The primary concern is that the model could be too focused on the technical and mathematical aspects of the data, providing explanations that are not understandable to a business user. The dataset integration tackles this exact problem by grounding the explanations in pragmatic advice that is coherent for the end user.

5.4 Multimodal approach

For a better understanding of the model's behavior and to provide a more complete explanation along with the metrics, the prompt includes base64-encoded images generated by the algorithm

during the analysis. This is particularly useful to recognize potential violations of data assumptions. For example, when using the linear regression model, the images of the residuals can be used to identify potential non-linear patterns in the data, which could be a sign of a poor model fit. The inclusion of the images in the prompt allows the model to provide a more complete explanation, taking into account not only the numerical data but also the visual information provided by the images.

5.5 Results

The prompt engineering process yielded satisfactory results. The model, following the instructions provided in the prompt, is able to generate explanations that are coherent with the data and manages to provide insights that are in line with business common sense.
The explanations are concise and focused on the most relevant features, avoiding the inclusion of irrelevant information. The model is also able to highlight any discrepancies between the data and user expectations, providing possible interpretations and corrective actions. Overall, the prompt engineering process has been successful in guiding the model to generate explanations that are useful and understandable for business users.
On some occasions, the model has shown contradictory behavior, for example saying that a certain feature is relevant in a first part and then saying the opposite. This is probably due to the fact that the models used for the analysis are quite small, and the task is quite complex.
However, the overall quality of the explanations is quite good, and the model is able to provide useful insights for the user.

Chapter 6

Conclusion

In this final chapter, we will expose the final state of the project and the evaluation of its outcomes.

6.1 Internship conclusion

The project reached a satisfactory final state. All planned stages were completed and project goals were achieved.

Goal	Description	Status
N-01	Complete and profound comprehension of the chosen algorithms, their mathematical foundations and their internal knowledge representation.	**Achieved**
N-02	Comprehension of interpretability and explainability techniques in machine learning.	**Achieved**
N-03	Comprehension of algorithms design techniques.	**Achieved**
N-04	Prompt generation for Large Language Models to generate human-readable explanations.	**Achieved**
N-05	Comprehension and application of Prompt Engineering principles in the context of XAI.	**Achieved**
D-01	Creation of a metric to evaluate the interpretability and explainability of the analyzed algorithms.	**Not Achieved**
D-02	Automatization of the analysis pipeline from dataset to LLM requests.	**Achieved**
O-01	Application in real-world scenarios of the interpretability and explainability of Machine Learning models.	**Partially Achieved**
O-02	Interpretability and explainability study of semi-supervised and unsupervised algorithms.	**Not Achieved**

Table of project goals final state.

The project has provided a comprehensive analysis of the interpretability and explainability of machine learning algorithms, with a practical implementation of an analysis system backed by a LLM that is able to generate human-readable explanations of algorithmic behaviors and predictions.

The details of the goal assessment are as follows:

- The goal D-01, which was to create a metric to evaluate the interpretability and explainability of the analyzed algorithms, was not achieved due to the nature of the task. It was not possible to create a metric that could be applied to all the algorithms analyzed, as the interpretability and explainability of each algorithm is highly dependent on the specific context and data used, making any single metric too generic to be useful.
- The goal O-01, which was to apply the analysis system in real-world scenarios, was only partially achieved due to the limited availability of suitable datasets. More on that on Section 6.4.
- The goal O-02, which was to study the interpretability and explainability of semi-supervised and unsupervised algorithms, was not achieved due to time constraints and preference in focusing on the supervised ones.

Overall, the project has been successful in achieving its objectives and providing a valuable addition to the personal and professional growth of the author, as well as its possible applications in business contexts.

6.2 Product final state

The results are satisfactory; the system aligns with the initial goal of making a flexible and adaptable system that can be applied to a wide range of algorithms and datasets, providing business users valuable insights.

Requirement	Description	Status
RFN-1	The system must allow the user to choose the dataset to analyse	**Achieved**
RFN-2	The system must allow the user to choose the algorithm for the analysis	**Achieved**

Requirement	Description	Status
RFN-3	The system must allow the user to choose the level of detail for the analysis (example SHAP/ No-SHAP)	**Achieved**
RFN-4	The system must allow the user to choose whether to execute the LLM analysis or not	**Achieved**
RFN-5	The system must allow the user to access the result of the analysis in a clear and organized way	**Achieved**
RFN-6	The system must allow the user to know the reason in case of an error during the execution of the analysis	**Achieved**
RFN-7	The system must allow the user to run multiple analyses to compare the performances of different algorithms and techniques	**Achieved**
RFO-1	The system should be accessible via graphical user interface (No-CLI)	**Achieved**
RQN-1	The system should be open to the use of different datasets, algorithms	**Achieved**
RQN-2	The system should be open to the use of different explainability and analysis techniques	**Achieved**
RQN-3	The system should be open to the use of different LLMs for the analysis	**Achieved**
RQD-1	The system should process the data and execute the analysis in a reasonable time frame, including the LLM analysis the pipeline should not take more than 15 minutes to execute	**Partially Achieved**

Table of requirements state for the analysis pipeline.

As shown in the table, the system is able to satisfy all the functional requirements. The only partially achieved requirement is RQD-1, which is related to the execution time of the analysis pipeline. The execution time is highly dependent on the LLM, as the infrastructure used for

the model is not under the control of the developer and is not currently optimized for running multiple models in parallel.

For complete tracking of the requirements, a requirements traceability matrix has been created.

Requirement	Module	Function
RFN-1	*selector.py*	*select_dataset()*
RFN-2	*selector.py*	*select_algorithm(task_type:TaskType)*
RFN-3	*selector.py*	*select_options()*
RFN-4	*selector.py*	*select_options()*
RFN-5	*orchestrator.py*	*export(self, model)*
RFN-6	Every module	Exception handling and logging
RFN-7	*selector.py*	*select_analysis_type()*
RFO-1	*app.py*	

Table of requirements state for the analysis pipeline.

6.3 Results

The analysis of the algorithms has provided valuable insights about their limitations and strengths, enabling a better understanding of how to improve their performance and interpretability. A comprehensive comparison of the algorithms is available in Section 3.9.

The integration of mathematical results, dataset context, and the analysis provided by the LLM enabled a more complete understanding of algorithmic behaviors. This justification closed the gap between the complexity of the algorithms and business users, providing clear and simple explanations and enabling informed decisions based on the results.

The results are generally satisfactory. This project provided an opportunity to explore the potential of LLMs in the context of explainable machine learning and to provide a practical implementation of a system, applying the knowledge built during the academic experience.

Moreover, the project offered a glimpse into the working world, exploring advanced technical aspects in a productive and professional environment.

6.4 Limitations and future work

The main limitation of the project is the limited impact of images. The pure mathematical nature of the plots and the complexity of the algorithms make it difficult to extract valuable insights from them, especially for business users. Researching techniques for better utilization of images is surely an aspect to improve. A first idea is the integration of a more precise description of the images, or perhaps supporting them with raw data. This is especially important for better identifying patterns and insights. This would require a deep study of how to extract the most relevant information from the images, if this is possible [20].

An important improvement would be to enhance efficiency as mentioned in Section 6.1, reducing the execution time. In the current version, the pipeline has two different ways to request the LLM: an LLM-wise and an algorithm-wise method. However, more sophisticated ways to navigate the technical limitations of the infrastructure could be explored.

Bibliography

[1] "Student Placement & Salary Dataset (Skills based)." [Online]. Available: https://www.kaggle.com/datasets/amaymishra11/student-placement-and-salary-dataset-skills-based

[2] "Life Expectancy (WHO)." [Online]. Available: https://www.kaggle.com/datasets/kumarajarshi/life-expectancy-who

[3] "Heart Failure Prediction Dataset." [Online]. Available: https://www.kaggle.com/datasets/fedesoriano/heart-failure-prediction

[4] Christoph Molnar, *Interpretable Machine Learning: A Guide for Making Black Box Models Explainable*. Leanpub, 2025.

[5] Zizhao Hu, Mohammad Rostami, and Jesse Thomason, "Expert Personas Improve LLM Alignment but Damage Accuracy: Bootstrapping Intent-Based Persona Routing with PRISM," 2026.

[6] "Which Table Format Do LLMs Understand Best?." [Online]. Available: https://www.improvingagents.com/blog/best-input-data-format-for-llms/

[7] Jasonn Wei *et al.*, "Chain of Thought Prompting Elicits Reasoning in Large Language Models," 2023.

[8] Takeshi Kojima, Shixiang Shane Gu, Machel Reid, Yutaka Matsuo, and Yusuke Iwasawa, "Large Language Models are Zero-Shot Reasoners," 2023.

[9] Taowen Wang *et al.*, "M^2PT: Multimodal Prompt Tuning for Zero-Shot Instruction Learning," 2024.

[10] "P-Value: What It Is, How to Calculate It, and Examples." [Online]. Available: https://www.investopedia.com/terms/p/p-value.asp

[11] "Major Kernel Functions in Support Vector Machine (SVM)." [Online]. Available: https://www.geeksforgeeks.org/machine-learning/major-kernel-functions-in-support-vector-machine-svm/

[12] Michal Mikulasi, "What Is the Sigmoid Kernel? (And Why It Feels a Bit Like Deep Learning)." [Online]. Available: https://medium.com/@michalmikuli/what-is-the-sigmoid-kernel-and-why-it-feels-a-bit-like-deep-learning-aa0210055c4e

[13] Lior Rokach and Oded Maimon, *Data Mining with Decision Trees: Theory and Applications*. World Scientific Publishing, 2008.

[14] -"Oded Maimon" -"Lior Rokach", *Decomposition methodology for knowledge detection*. World Scientific Publishing, 2010.

[15] “Scikit-learn Documentation.” [Online]. Available: https://scikit-learn.org/stable/supervised_learning.html

[16] Tianqi Chen and Carlos Guestrin, “XGBoost: A Scalable Tree Boosting System,” *KDD '16*, 2016.

[17] -"Cristina Cornelio" -"Sanjeeb Dash" -"Vernon Austel" et al., “Combining data and theory for derivable scientific discovery with AI-Descartes,” 2023, *Nature Communications*.

[18] Robert C. Martin, “Design principles and design patterns,” 2002. [Online]. Available: https://objectmentor.com/resources/articles/Principles_and_Patterns.pdf

[19] Robert C. Martin, *Clean Code: A Handbook of Agile Software Craftsmanship*. Pearson, 2008.

[20] -"Mohammad Asadi" -"Jack W. O'Sullivan" -"Fang Cao" -"Tahoura Nedaee" -"Kamyar Rajabalifardi" -"Fei-Fei Li" -"Ehsan Adeli" -"Euan Ashley", “MIRAGE: The Illusion of Visual Understanding,” 2026, *arXiv*.

[21] Zhen "Leo" Liu, *Artificial Intelligence for Engineers: Basics and Implementations*. Springer, 2024.

[22] Candice Bentéjac, Anna Csörgő, and Gonzalo Martínez-Muñoz, “A Comparative Analysis of XGBoost,” *Artificial Intelligence Review*, vol. 54, no. 3, 2019.

[23] Christopher M. Bishop, *Pattern Recognition and Machine Learning*. Springer, 2006.

[24] G. James, D. Witten, T. Hastie, R. Tibshirani, and J. Taylor, *An Introduction to Statistical Learning: with Applications in Python*. Springer, 2023.

[25] V. K. Xidong Wu, *Top 10 Algorithms in Data Mining*. CRC Press, 2009.

[26] “SHAP Documentation.” [Online]. Available: https://shap.readthedocs.io/en/latest/

[27] Xavier Amatriain, “Prompt Design and Engineering: Introduction and Advanced Methods,” 2024.

[28] -"Gabriel Kronberger" -"Michael Kommenda" -"Stephan Winkler" -"Bogdan Burlacu", *Symbolic Regression*. CRC Press, 2025. [Online]. Available: https://www.researchgate.net/publication/382152886_Symbolic_Regression

Glossary

***AI* – Artificial Intelligence**: Field of computer science that focuses on creating systems capable of performing tasks that typically require human intelligence, such as learning, reasoning, problem-solving, and understanding natural language. ii, 1

***categorical features* – Categorical Features**: Features in a dataset that represent discrete categories or groups, rather than continuous numerical values, and often require special handling in machine learning algorithms. 5, 15, 17, 22, 34, 34, 34, 39

***classification* – Classification**: A type of machine learning problem in which the goal is to predict discrete class labels based on input features. ii, 1

***CLI* – Command Line Interface**: A text-based interface that allows users to interact with a computer system by typing commands, rather than using graphical elements. 59

***computational complexity* – Computational Complexity**: A measure of the amount of computational resources (time and space) that an algorithm requires to run, often expressed in terms of the size of the input data. 5, 14, 15, 15

***correlation matrix* – Correlation Matrix**: A table showing the correlation coefficients between pairs of features in a dataset, used to identify relationships and potential multi-collinearity. 16, 23

***XAI* – Explainable Artificial Intelligence**: A field of artificial intelligence that focuses on creating models and systems that can provide clear and understandable explanations for their decisions and actions. ii, ii, 1, 10

***FN* – False Negatives**: Instances incorrectly predicted as negative. 11, 11

***FP* – False Positives**: Instances incorrectly predicted as positive. 11, 11

***GD* – Gradient Descent**: An optimization algorithm used to minimize the objective function in machine learning models by iteratively adjusting the model's parameters in the direction of the steepest descent. 14, 15, 15, 16, 21, 21

***GUI* – Graphical User Interface**: A user interface that allows users to interact with a computer system through graphical elements such as windows, icons, and buttons, rather than text-based commands. 59, 61

***LLM* – Large Language Model**: A type of artificial intelligence model that is trained on vast amounts of text data to understand and generate human-like language. ii, ii, 1, 8, 8, 8, 8, 10, 11, 61, 62, 62, 63, 66, 68, 71, 71, 71, 76, 77, 78, 78, 79

***LP* – Linear Programming**: A mathematical optimization technique used to find the best outcome in a model whose requirements are represented by linear relationships, often used in machine learning for training linear SVMs and other models. 27

***MAE* – Mean Absolute Error**: A common metric for evaluating the performance of regression models, calculated as the average of the absolute differences between predicted and actual values. 14, 51

***margin* – Margin**: The distance between the decision boundary (hyperplane) and the closest data points in a Support Vector Machine, which the algorithm seeks to maximize for better generalization. 25, 25, 25

***ML* – Machine Learning**: A subset of artificial intelligence that involves training algorithms to learn patterns from data and make predictions or decisions without being explicitly programmed. ii, 1, 6

Nyström method: A technique used to approximate large kernel matrices in machine learning, which allows for efficient training of models like Support Vector Machines on large datasets by sampling a subset of the data and using it to construct a low-rank approximation of the kernel matrix. 27

***objective function* – Objective Function**: A mathematical function that an algorithm optimizes during training, which defines the goal of the learning process and guides the model's adjustments to minimize error or maximize performance. 5, 42

outlier: A data point that deviates significantly from the majority of the data, which can have a disproportionate influence on machine learning models and may require special handling. 17, 29

***PE* – Prompt Engineering**: The process of designing and optimizing prompts to effectively communicate with language models and elicit desired responses. 1

***RFF* – Random Fourier Features**: A technique used to approximate kernel functions in machine learning, allowing for efficient training of models like Support Vector Machines on large datasets by mapping input data into a higher-dimensional space using random Fourier transformations. 27

***regression* – Regression**: A type of machine learning problem in which the goal is to predict continuous numerical values based on input features. ii, 1

***RMSE* – Root Mean Squared Error**: A common metric for evaluating the performance of regression models, calculated as the square root of the average of the squared differences between predicted and actual values. 14, 51

***SMO* – Sequential Minimal Optimization**: An algorithm used to solve the optimization problem in Support Vector Machines by breaking it down into smaller subproblems that can be solved analytically, which allows for efficient training of SVMs on large datasets without the need for complex numerical optimization techniques. 27

***SHAP* – SHapley Additive exPlanations**: A method for explaining the output of machine learning models by attributing the prediction to each feature. ii, 47, 57, 59

***SGD* – Stochastic Gradient Descent**: An optimization algorithm that updates the model's parameters using a randomly selected subset of the training data, which can lead to faster convergence and better generalization. 15, 21, 27

***TN* – True Negatives**: Instances correctly predicted as negative. 11, 11

***TP* – True Positives**: Instances correctly predicted as positive. 11, 11